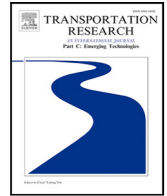




Contents lists available at ScienceDirect

Transportation Research Part C

journal homepage: www.elsevier.com/locate/trc

Railcar itinerary optimization in railway marshalling yards: A graph neural network based deep reinforcement learning method

Hongxiang Zhang^{a,d}, Gongyuan Lu^{a,b,c}, Yingqian Zhang^d, Andrea D'Ariano^e,
Yaoxin Wu^{d,*}

^a School of Transportation and Logistics, Southwest Jiaotong University, 610031, Chengdu, China

^b Comprehensive Transportation Key Laboratory of Sichuan Province, 610031, Chengdu, China

^c National United Engineering Laboratory of Integrated and Intelligent Transportation, 610031, Chengdu, China

^d Department of Industrial Engineering & Innovation Sciences, Eindhoven University of Technology, 5600 MB Eindhoven, The Netherlands

^e Department of Civil, Computer Science and Aeronautical Technologies Engineering, Roma Tre University, Rome, 00146, Italy

ARTICLE INFO

Keywords:

Railway marshalling yard
Train makeup problem
Hump sequencing
Classification track assignment
Graph neural network
Deep reinforcement learning

ABSTRACT

The goal of Railcar Itinerary Optimization in Marshalling Yards (RIO-MY) is to achieve an effective integrated operation plan for both train shunting operations and train makeup, with the aim of minimizing the railcar dwell time in the railway marshalling yard. Due to complex interdependent decisions in the disassembly and assembly process of trains, conventional optimization methods for the problem face challenges in addressing the dynamic nature of traffic in the marshalling yard and offering highly efficient solutions. This paper introduces a novel approach to the RIO-MY problem using a graph neural network based deep reinforcement learning method. First, we model the solving process of RIO-MY as a Markov decision process, utilizing a tripartite graph to represent the operational state of a marshalling yard. Then we design a novel tripartite graph isomorphism network (TGIN) to learn informative embeddings on the graph, which are exploited to reason out the joint action to simultaneously decide on hump sequencing and classification track assignment. The TGIN based policy network is trained by the proximal policy optimization algorithm, with a reward tailored to well estimate railcar dwell time for each state. Moreover, we develop a discrete-event simulation of operations in the railway marshalling yard, which serves as the reinforcement learning environment and integrates typical heuristic rules of outbound train assembly and shunting locomotive scheduling. Extensive experiments on two real-world railway marshalling yards demonstrate that the proposed method outperforms conventional heuristic algorithms. Moreover, it achieves competitive performance to the mixed integer nonlinear programming model with significantly less computational time. In addition, the trained policy networks can favourably generalize to scenarios that are unseen during training and effectively handle disturbances in the train disassembly process.

1. Introduction

Railway marshalling yards serve as critical hubs of railway freight transportation, since their operation substantially influences the efficiency of the whole railway network. Specifically, the marshalling yards play a vital role in disassembling inbound trains into

* Corresponding author.

E-mail addresses: hongxiang@my.swjtu.edu.cn (H. Zhang), lugongyuan@swjtu.edu.cn (G. Lu), YQzhang@tue.nl (Y. Zhang), andrea.dariano@uniroma3.it (A. D'Ariano), y.wu2@tue.nl (Y. Wu).

<https://doi.org/10.1016/j.trc.2024.104970>

Received 20 December 2023; Received in revised form 8 December 2024; Accepted 10 December 2024

Available online 20 December 2024

0968-090X/© 2024 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

individual railcars and subsequently assembling these railcars to create new outbound trains. This disassembly and assembly process ensures the seamless transportation of railcars throughout the railway network, offering the following benefits: (1) The freight transport between numerous origin–destination pairs is realized, without building innumerable train services that directly go from origins to destinations but unnecessarily consume the capacity of the railway network. (2) The unbalanced freight demands from an origin to multiple destinations can be easily accommodated. Despite these benefits, the disassembly and assembly in marshalling yards involve a complex procedure, carried out through a series of shunting operations. This process contributes significantly to the dwell time of each railcar, i.e., the time between the arrival of its inbound train and departure of its outbound train, accounting for approximately 60% of their total transport time (Zhao and Dick, 2023). Therefore, it is crucial to improve the shunting operation efficiency to reduce the railcar dwell time, fostering more productive railway freight transportation.

During the dwell time, railcars undergo the itinerary constituted by the specific operations and spatial–temporal movements in the marshalling yard. In this paper, we study the Railcar Itinerary Optimization in the Marshalling Yard (RIO-MY) problem to reduce railcar dwell time. Specifically, the RIO-MY problem aims to design an integrated operation plan for train shunting operation and train makeup. The train shunting operation plan involves multiple decisions for disassembly and assembly at the operational level, such as hump sequencing, shunting locomotive scheduling, classification track assignment, and assembly sequencing. The train makeup plan involves the decisions of car-to-train assignment at the tactical level, i.e., assigning railcars from inbound trains to form outbound trains. Therefore, the RIO-MY problem inevitably entails complex and interdependent decisions. On the one hand, diverse interdependencies exist among different shunting operations at the operational level, e.g., the classification track assignments to be completed in a certain period are directly determined by the hump sequencing. On the other hand, the cross-level interdependency between the operational train shunting and the tactical train makeup also needs to be taken into account, since efficient train shunting operations tend to achieve train makeup plans with short dwell time of railcars between inbound trains and assigned outbound trains. Furthermore, the integrated operation plan must comply with lots of practical regulations and restrictions, such as the destination formation requirement of an outbound train, the maximum parking capacity of a classification track, etc., further increasing the complexity of the RIO-MY problem.

The existing literature addresses diverse RIO-MY problems, with various problem characteristics and optimization objectives. These variations come from the diverse contexts, procedures, and regulatory frameworks governing railway operations in different countries. Some studies focus on various aspects of the railcar itinerary, e.g., the humping sequence in the shunting operations, the straightforward train makeup plan beyond the shunting operations, etc. (Boysen et al., 2016; Jaehn et al., 2015b; Li et al., 2016). Despite these variations, the literature has indicated that mathematical programming is broadly applicable to optimization problems in railway marshalling yards, often delivering good solutions (Boysen et al., 2012). However, the mathematical programming approach has several limitations. It usually suffers from heavy workloads in constructing mathematical models that closely mimic real-world scenarios, and lacks flexibility in handling changing parameters under dynamic contexts (Shi and Zhou, 2015; Jaehn et al., 2015b). Furthermore, heuristics or relaxation/decomposition for models have to be employed to cope with the high computational costs required to obtain high-quality solutions (Shi and Zhou, 2015; Haahr and Lusby, 2016; Raut et al., 2019). These issues hinder the application of mathematical programming in real-world scenarios. Recent years have witnessed the emergence of deep reinforcement learning (DRL), demonstrating promising capability in solving combinatorial optimization problems such as job-shop scheduling problems (Zhang et al., 2020a; Song et al., 2023), train (re)scheduling problems (Li and Ni, 2022; Ying et al., 2020), vehicle routing problems (Wu et al., 2023; Zhou et al., 2023), etc. Some works have shown the enormous potential of DRL methods that can attain comparable performance to highly specialized algorithms for specific optimization problems (Kool et al., 2018; Kwon et al., 2020; Ma et al., 2022; Lei et al., 2022; Li et al., 2021). By modelling the solving procedure for a specific problem as a Markov decision process (MDP), the DRL methods are able to automatically discover effective policies to obtain favourable solutions without too much manual work on mathematical formulation, uncertainty modelling, parameter tuning, etc. The policies in the DRL methods are often parameterized by graph neural networks (GNNs), which capture informative representations (i.e. embeddings) of nodes and edges from graph encoded data via iterative message passing and aggregation (Cappart et al., 2023; Wu et al., 2020). In doing so, GNN empowered policy learning can encourage the DRL agent to better understand the environment and reason out effective solutions. Despite the promising outcome, existing DRL methods are focused on simple combinatorial optimization problems and lack the applications to the RIO-MY problem, which is a complex yet valuable real-world use case.

In this paper, we propose a novel GNN based DRL method to solve the RIO-MY problem in the railway marshalling yard. First, we formulate the solving process of the RIO-MY problem as an MDP, in which an agent iteratively executes joint actions (i.e. decisions) on hump sequencing and classification track assignment. Then, we design a tripartite graph with three node sets signifying inbound trains, outbound trains, and classification tracks, which encode states to reflect the traffic dynamics in the railway marshalling yard. Next, we come up with a novel tripartite graph isomorphism network (TGIN) to process the graph encoded state and generate node embeddings. Based on embeddings, the other decision-making network (DMN) computes the probabilities of candidate joint actions at each step, from which the action is selected in a sampling manner. To well model state transitions after action executions, we develop a discrete-event simulation of the railcar itinerary, which implements heuristic rules for other operations (e.g. assembling outbound trains and scheduling shunting locomotive), as well as realistic regulations and restrictions in the railway marshalling yard. Finally, the proximal policy optimization (PPO) algorithm is used to train the TGIN based policy network, where the railcar dwell time estimation based reward function is leveraged to derive the policy optimization formula. With the power of policy network and PPO, the DRL agent can automatically attain an effective policy to deliver favourable joint actions, without heavy manual work on feature engineering and heuristics design. In summary, this paper contributes in the following aspects:

- To the best of our knowledge, this paper presents the first DRL method to solve the railcar itinerary optimization problem in railway marshalling yards. The proposed method has the potential to offer fresh perspectives on the practical implementation of DRL to solve real-world complex combinatorial optimization problems, which automate the policy learning for critical decisions, without the need for intensive labor on specializing and tuning heuristic algorithms.
- The solving process of RIO-MY is modelled as a novel MDP to facilitate our DRL method to automate the algorithm design for the problem. We employ a novel tripartite graph for state encoding, define a joint action to simultaneously decide on the humping sequence and classification track assignment, and design a railcar dwell time estimation based reward function. The TGIN based policy network is used to parameterize the policy and trained to derive effective actions from the graph encoded states.
- Extensive experiments show that the proposed method outperforms the commonly used heuristic algorithms. In comparison to the decomposed mixed integer nonlinear programming (decom-MINLP) model solved by Gurobi, the average gap of our high-quality solutions is only 3.92% in a set of testing instances, while the average solving time of each instance is only 9 s (vs. 46 s by Gurobi).
- The trained policy has strong generalization in solving instances with various characteristics of inbound and outbound trains, which have never been seen during training. The trained policy is also verified to be able to deliver new solutions in real time when a disturbance occurs in the disassembly process of an inbound train. These experimental results manifest the potential of our DRL method to be applied in disparate railway marshalling yards with diverse layouts and varying dynamics.

We arrange the remainder of this paper as follows. Section 2 reviews the problems in the context of railway marshalling yards and the application of DRL methods in the train traffic management. Section 3 describes the RIO-MY problem and provides its mixed integer nonlinear programming model. Section 4 presents our DRL method with the GNN to solve the RIO-MY problem. In Section 5, we conduct extensive experiments with the problem settings in two real-world railway marshalling yards, to show the performance, efficiency, and generalization capability of our method. Finally, we conclude this paper and outline the potential further research in Section 6.

2. Literature review

In this section, we first review the railcar itinerary related optimization problems in railway marshalling yards. Then, we provide a brief review of research that uses reinforcement learning methods to solve combinatorial optimization problems over graphs. Finally, we recap the reinforcement learning methods applied in train traffic management.

2.1. Railcar itinerary optimization in the marshalling yard

In the last few decades, the RIO-MY problems have been widely studied, with the aim to ameliorate the decision on the arrangement and utilization of trains, railcars, locomotives, and infrastructures. These problems are mainly categorized into four main branches in Boysen et al. (2016), including the cut generation problem, the train makeup problem, the railcar classification problem, and the outbound track assignment problem. Since our target RIO-MY problem in this work is more related to the train makeup problem and railcar classification problem, we elaborate on the literature on these two problems in the following content, and refer interested readers to the other sub-problems in Boysen et al. (2012).

2.1.1. Railcar classification problem

The railcar classification problem aims to optimize one or more train shunting operations, e.g., the hump sequencing, shunting locomotive scheduling, classification track assignment (Ruf and Cordeau, 2021). Two classification strategies, i.e., single-stage sorting and multi-stage sorting, are commonly used, according to Boysen et al. (2012). The difference between them is whether a railcar can be pulled back from the classification yard and humped more than once. Despite quite a few multi-stage sorting based studies (Maue and Nunkesser, 2009; Jacob et al., 2011; Bohlin et al., 2012, 2016, 2018), we concentrate on the literature on the single-stage sorting strategy in this paper, since it is applied in the vast majority of railway marshalling yards in China, which are deployed in our target problems. As a seminal research, Yagar et al. (1983) optimize the humping sequence to minimize throughput costs of a marshalling yard, which are influenced by the terminal dwell time of railcars. Dahlhaus et al. (2000) try to assemble railcars in the desired order to form outbound trains with the minimal number of classification tracks. Di Stefano and Koči (2004) study a similar problem in a depot for passenger trains. Jaehn et al. (2015a) adopt two heuristics and two exact branch-and-bound algorithms to minimize the outbound train delays by hump scheduling. Kraft (2000, 2002b) propose a strategy called extended single-stage sorting, in which only a few partial railcars can be rehumped. Following this setting, Zhang et al. (2018) optimize the classification track assignment for railcars to minimize the number of tracks needed for parking rehumped railcars. Considering that the train makeup plan is given, the extended single-stage sorting can be treated as a guarantee measure to realize it. Otherwise, some tolerance is needed, such as the permission of train delays (Yagar et al., 1983; Jaehn et al., 2015a) and the neglect of the predetermined train makeup plan (Kraft, 2000).

2.1.2. Train makeup problem

The train makeup problem (a.k.a. wagon-flow allocation or car-to-train assignment problem) involves optimizing the way that the railcars in inbound trains are assigned to outbound trains. Considering the priority of railcars, Kraft (2002a) design a train makeup

problem to bypass the situation where railcars with high priorities are blocked by the ones with low priorities on classification tracks. Bektaş et al. (2009) attempt to redistribute empty railcars into different outbound trains to mitigate the total dwell time of railcars. This train makeup problem is modelled as a maximum weighted matching problem and solved by an exact Hungarian Algorithm. Afterwards, Li et al. (2014) study a similar problem in the context of railway marshalling yards in China, where railcars can be assigned to multiple possible outbound trains, and more restrictions (e.g. weight, length, and time) are placed on outbound trains. The authors develop a mathematical optimization model for the problem and then solve it using a heuristic algorithm. On the other hand, to maximize the priority values of loaded cargo, Boysen et al. (2016) optimize the railcar assignment from inbound trains to outbound trains without timetable restrictions. In this train makeup problem, it is assumed that all the shunting activities can be completed on time as required, according to a general situation in European countries (Boysen et al., 2013). This research also states that the solutions to the train makeup problem determine the given jobs for the railcar classification problem. If only unfavourable solutions are obtained for these jobs by solving the railcar classification problem, the given train makeup cannot be achieved and another alternative train makeup plan should be provided. Therefore, the isolated research on the only train makeup problem may not be applicable, since its solutions are meanwhile dependent on how the railcar classification problem is solved.

2.1.3. Integrated modelling

Considering the interdependency between the train makeup problem and the railcar classification problem, it is natural to study them together in a model. He et al. (2000) describe an integrated approach for optimizing not only the train makeup plan but also the humping and assembly operations. As a follow-up study, He et al. (2003) extend the problem to include the scheduling of shunting locomotives. But the classification track assignment for railcars is ignored in the above research. Similarly, Li et al. (2016) consider the hump sequencing, assembly sequencing, and train makeup problem, and propose a harmony heuristic algorithm to solve the problem. To minimize the weight departure time, Jaehn et al. (2015b) study the problem that assigning railcars to form single-destination outbound trains without specified railcar orders. The car-to-train assignment is determined by the humping sequence in their framework since a railcar can be assigned to multiple outbound trains. Against the background of Problem Solving Competition 2014 held by INFORMS RAS, Haahr and Lusby (2016) come up with a solution to sequentially cope with the hump sequencing, classification track assignment, and pull-out shunting locomotive scheduling. Raut et al. (2019) work on the same problem and propose a rolling horizon based algorithm to solve it by modelling all three sub-problems in a single optimization framework. In addition, a crew scheduling problem for shunting activities is further included as another optimization target (Ruf and Cordeau, 2021).

In summary, the above research always develops mathematical programming models for the railcar classification problem. Despite the objective functions of these mathematical models usually determined by the various realistic demands from research background, they generally focus on saving time for trains or resources for the marshalling yard, such as minimizing departure train delays, minimizing the number of required classification tracks, reducing railcar dwell time in the marshalling yard, etc. These objective functions are affected by many operational decisions, including classification track assignment, car-to-train assignment, hump sequencing, etc. Thereby, decision variables have to cover multiple aspects of train operation in the marshalling yard. For example, binary variables are necessary for assignment decisions of railcars from inbound trains to outbound trains; assigning classification tracks for railcars also needs binary variables to model decisions; time should be discretized into numerous integer variables if the model must determine the beginning time of disassembly or assembly. The various objective functions and the large number of decision variables lead to large-scale models and also difficulties in efficiently solving the models using exact algorithms. Hence, in the above research, the models are usually solved with heuristic algorithms, which are proven to be able to obtain good solutions. However, such a solving paradigm still faces some challenges, including inferior flexibility in handling dynamic train operation parameters, low computational efficiency, and inability to deal with train operation disturbance. It inspires us to apply the DRL method, by which we train the policies for specific decisions in shunting operations in a data-driven manner. The trained policies can rapidly reason out high-quality solutions in real-time and work well with dynamics in the environment. Particularly, we aim to automate the learning of policies for hump sequencing and classification track assignment, and tackle the other shunting operations with practical heuristic rules. In doing so, our method can deliver efficient railcar itineraries, which leads to short railcar dwell time and favourable train makeup plans.

2.2. Reinforcement learning over graphs for combinatorial optimization problems

In recent years, reinforcement learning methods, especially those incorporated with graph neural networks, have shown promising capabilities in solving combinatorial optimization problems (Khalil et al., 2017). As stated in Mazyavkina et al. (2021), when using reinforcement learning to solve combinatorial optimization problems, the graph neural network is often used as the encoder for states of MDP, i.e. a parametric function that encodes the input states and outputs a numerical vector. The encoded vectors provide plenty of information for reinforcement learning agents to automate a good policy. Such a GNN based manner has been proven to work very well for various combinatorial optimization problems, including Travelling Salesman Problem (TSP, Bello et al., 2016; Cappart et al., 2023; Drori et al., 2020), Vehicle Routing Problem (VRP, Wu et al., 2021; Lin et al., 2024; Xu et al., 2021), Maximum Cut Problem (MCP, Cappart et al., 2019; Tang et al., 2020), Bin Packing Problem (BPP, Hu et al., 2017; Duan et al., 2018), Job Shop Scheduling Problem (JSSP, Zhang et al., 2020b; Park et al., 2021; Smit et al., 2024), etc. For example, in Zhang et al. (2020b), a disjunctive graph is used to model JSSP and represent states of the corresponding MDP. A powerful GNN encodes all nodes in the disjunctive graph for automatic policy learning by reinforcement algorithms. The decision problems in a railway marshalling yard have similarities to JSSP and VRP (Ruf and Cordeau, 2021). A railcar must be processed in a predetermined

order, including humped to the hump, disassembled from the train, parked on a classification track, and assembled to form a new train. This process, each task in the process, and train shunting locomotives or classification tracks to execute tasks can be seen as jobs, operations, and machines of a JSSP. The order in which the resources (shunting locomotives) visit requests (trains and railcars) is also reminiscent of VRP. Despite such similarities, the RIO-MY problem cannot be classified as one of the standard combinatorial optimization problems due to complex practical constraints and interdependent decisions. In this paper, we propose the first GNN-based DRL method to solve the RIO-MY problem, with much effort in specializing the MDP formulation and GNN design.

2.3. DRL applications in train traffic management

In the last section, we recognize that reinforcement learning has been applied to solve combinatorial optimization problems. However, only a few works are applied in train traffic management. We brief them in the following content.

2.3.1. Train operation control

Train operation control is an optimization problem to control train running processes with purposes related to energy-saving, passenger comfort, punctuality, etc. (Su et al., 2021; Zhang et al., 2021; Wang et al., 2023b). As representative research, Yin et al. (2014) propose two train operation control algorithms based on the expert system and reinforcement learning to optimize the train speed curve and reduce the energy consumption of train operation. Zhu et al. (2022) define two different train operation states according to train running time and energy distributions. A Q-learning method is proposed to obtain a high-quality strategy for energy distribution. Chen et al. (2023) propose a train control method based on the deep Q-Network algorithm, with the objective of automating the train operation. The authors consider the requirements of energy-saving, safety, punctuality, and accurate parking. Despite the effectiveness of reinforcement learning, the problems studied in the above works are totally different from the RIO-MY problem.

2.3.2. Train (re)scheduling

The train scheduling and rescheduling problems under disruptions attract increasing research attention from the reinforcement learning domain. As an early work (Šemrov et al., 2016) develop a Q-learning based algorithm to optimize the timetable rescheduling on a single-track railway line for minimizing train delays. Khadilkar (2018) propose a scalable Q-learning algorithm for a similar problem. Zhu et al. (2020) further propose a Q-learning method for train rescheduling on a double-track railway line. Ying et al. (2020) adopt the deep deterministic policy gradient (DDPG), a commonly used reinforcement learning algorithm, to solve the train scheduling problem in a metro system. As a follow-up work, Ying et al. (2022) optimize both the train composition problem and train scheduling problem on a single metro line. Li and Ni (2022) address the train scheduling problem for mainline railways by a multi-agent deep reinforcement learning method. They treat each train as an agent, and merge the states, actions, and rewards in the mathematical formulation. Agasucci et al. (2023) design a graph convolution network (GCN) to estimate train delays, which are used in the deep Q-learning method for the train rescheduling optimization under disturbances.

2.3.3. Train unit shunting

The train unit shunting problem refers to the optimization of the shunting for train units in passenger train depots. A train unit is composed of a fixed number of coaches. One or more train units will form a passenger train. The train unit shunting problem contains two important sub-problems which are similar to the RIO-MY problem. One is the matching problem, which aims to assign train units from arrival trains to assemble departure trains according to the types of train units. The other is the parking problem, which aims to park arrival trains on tracks of the shunting yard. A parking strategy is required to provide unobstructed positions for departure. Among reinforcement learning methods for train unit shunting, (Peer et al., 2018) represent the state including various information on train units as an image. They define an action as the selection of a shunting track to park an arrival train or depart the train parking on it. Lee et al. (2020) model the MDP transition for train unit shunting based on discrete time intervals, which enables the DRL method to include time constraints of train service tasks on shunting tracks. Trepát Borecka et al. (2021) provide a multi-agent DRL framework, where actions are categorized into eight types to complete all shunting processes. Additionally, de Oliveira da Costa et al. (2020), van de Ven et al. (2019a) and van de Ven et al. (2019b) propose graph neural network based methods for rapid judgment of the feasibility of given shunting plans, which can be used in heuristics for efficiently solving the train unit shunting problem.

According to the literature, especially the works on solving train (re)scheduling problems by reinforcement learning, the most effort in applying DRL in train traffic management is put into manually crafting features to represent the states in the MDP. The potent ability of deep learning in the automatic extraction of informative representations has been paid little attention to. In our RIO-MY problem, much more information on the status of trains, railcars, and tracks has to be represented in the state. Many heterogeneous entities need to be described, which is much more complex than the above problems. Therefore, we opt to use graph neural networks to learn effective representations automatically, which assist in training favourable policies. As we know, this is a very early study on the application of DRL methods to solve decision-making problems in railway marshalling yards.

3. Problem statement

In this section, we first introduce the train operation processes and the required decisions in the railway marshalling yard. Then, a mathematical model is formulated to describe the RIO-MY problem that aims to reduce railcar dwell time. Finally, we

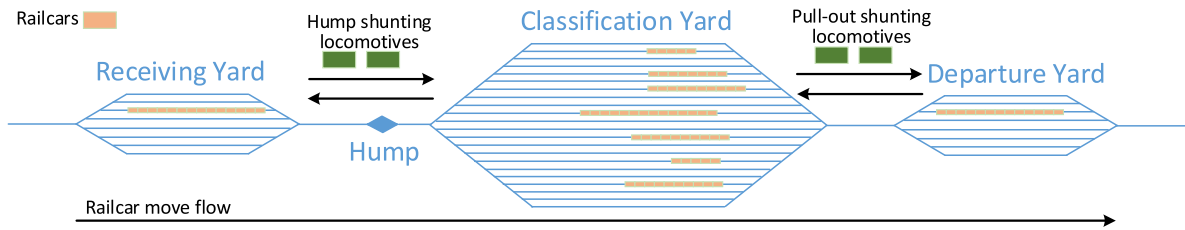


Fig. 1. The schematic layout of a standard railway marshalling yard.

opt to learn the crucial decisions in train shunting operations by DRL in our method, which otherwise could be inferior by simple heuristic algorithms. It should be noted that the target problem in this paper is described according to backgrounds, scenarios, and management regulations from railway marshalling yards in China, which we use to set up our experiments and evaluate the methods.

3.1. RIO-MY description

The main job in a railway marshalling yard is to disassemble inbound trains into railcars, and assign railcars to form new outbound trains. A typical layout of a marshalling yard is shown in Fig. 1. It comprises a receiving yard, a hump, a classification yard, and a departure yard. The receiving yard contains a set of berthing tracks for parking inbound trains. The hump is a raised, sloped section of track area between the receiving yard and the classification yard, which enables the disassembled railcars to roll into the classification yard by gravity. The classification yard contains parallel tracks connected to the hump for accumulating railcars. The departure yard comprises a set of berthing tracks used to park the formed outbound trains. A series of train operations are executed in the railway marshalling yards, which can be summarized in the following procedure. (1) **Arrival**: An inbound train arrives at the receiving yard and parks on a berthing track. A technical inspection is executed immediately on the train after its arrival to prepare it for the subsequent humping. (2) **Humping and Disassembly**: The inbound train is pushed by a hump shunting locomotive over the hump, and then disassembled into railcars consecutively. The disassembled railcars roll into the assigned classification track from the hump. (3) **Accumulation**: The railcars park on classification tracks until they are pulled out for assembly. The railcars on each classification track are in a queue, following the first-in-first-out principle. (4) **Assembly**: A pull-out shunting locomotive moves to the classification yard and assembles outbound trains with the railcars in classification tracks. Then every new outbound train is transferred by the pull-out shunting locomotive to the departure yard and then parks on a berthing track. (5) **Departure**: After a technical inspection, outbound trains depart at the scheduled time. The above procedure from arrival to departure is termed the railcar itinerary, which involves specific operations and spatial-temporal movement for railcars. Most of the operations relate to train shunting, which affects the efficiency of the railcar itinerary and thus the railcar dwell time, i.e., the duration from the arrival of a railcar (in an inbound train) to its departure (in an outbound train). Simultaneously, the train shunting operations have cross-level interdependency with the train makeup plan. Accordingly, we first elaborate on the four operational decisions in the train shunting operations and then the tactical decision in the train makeup plan, together with the relevant restrictions in practice.

- **Hump sequencing.** Hump sequencing involves decisions on the order, in which the inbound trains are disassembled, and the time when each inbound train starts to be humped and disassembled. In most railway marshalling yards in China, only one hump exists to disassemble one inbound train at a time. The hump sequencing should guarantee that all railcars are disassembled from inbound trains in time, so that they can be efficiently processed by subsequent operations for successful departures.
- **Classification track assignment.** The classification track assignment involves the decisions on which track should be assigned to a railcar for its parking after it is disassembled from the inbound train. The number of railcars on a classification track cannot exceed its capacity. A railcar has a destination and each classification track has its own preferred destination. The track whose preferred destination is identical to the destination of a railcar, named **prioritized classification track**, has the highest priority to be assigned to that railcar. In contrast, the other non-prioritized classification tracks are considered for the assignment only if the prioritized classification tracks do not have enough capacity. For each destination, it is set to be preferred by at least one classification track. In addition, only single-stage sorting is allowed, and the first-in-first-out (FIFO) principle is applied to roll in and pull out railcars on a classification track.
- **Assembly sequencing.** Assembly sequencing involves decisions on the order, in which the outbound trains are assembled, and the time when each outbound train starts to be assembled. The specific shunting operations for assembling an outbound train are affected by the railcar distributions in the classification tracks, which are determined by the train makeup plan and the classification track assignment. The assembly sequencing decisions are for the outbound trains with specified departure times in the given timetable. No assembly is needed for cancelled outbound trains. The assembly operations for multiple trains can be performed in parallel, since there are usually 2 to 4 leading tracks (tracks for transferring railcars between the classification yard and departure yard) to accommodate pulled-out railcars.

- **Shunting locomotive scheduling.** Shunting locomotive scheduling involves decisions on assigning hump shunting locomotives to disassemble inbound trains and pull-out shunting locomotives to assemble outbound trains, respectively. Obviously, they are highly related to the hump and assembly sequencing. In practice, the number of shunting locomotives is limited. In this paper, we follow a usual situation in real-world railway marshalling yards, i.e., two hump shunting locomotives and two pull-out shunting locomotives are available, concurrently. Only one locomotive is needed for a single disassembly or assembly task.

The above four types of operational decisions for shunting are highly intertwined, together influencing the railcar itinerary. However, railcar itineraries are running in a very complex environment. The above operational decisions always have complex interdependency on the high-level tactical decision, i.e., the train makeup plan. We further elaborate on the related decisions and constraints, which also interact with the above operational decisions.

- **Train makeup plan.** The train makeup plan determines railcars from inbound trains would be assigned to which outbound trains. It should consider the railcar destinations, number of railcars, and outbound train timetable. Specifically, each outbound train has a specified formation requirement of destinations. Only the railcars with destinations that satisfy the formation requirement of an outbound train can be assigned to it. Furthermore, each outbound train has minimum and maximum number limits of railcars. The number of railcars assigned to an outbound train must not exceed the maximum number limit. Meanwhile, an outbound train has a scheduled departure time in the given train timetable. Therefore, considering the necessary time of *assembly* and *departure*, the minimum railcar number limit must be reached before a specific latest assembly start moment to guarantee the outbound train can depart on time. Otherwise, the outbound train will be **cancelled**.

Various cross-level interdependencies exist between the train shunting operations and the train makeup plan. On the one hand, the train makeup plan specifies the start and end of the itinerary for a railcar, in which the assignment of a railcar to an outbound train naturally sets a time limit for the completion of shunting operations for the railcar. On the other hand, the profitable train makeup plan is significantly dependent on efficient shunting operations. If an inbound train is not disassembled in time or classification tracks are not reasonably assigned to railcars, the subsequent assembly and departure could be influenced, thereby inducing failed car-to-train assignments, cancelled outbound trains, and unfeasible train makeup plan. Hence, making high-quality decisions on shunting operations should take the cross-level interdependencies into account.

In summary, the operational decisions for train shunting and the tactical decisions for train makeup are dependent on each other, which together play significant roles in determining the efficiency of the railcar itinerary and thus the railcar dwell time. Considering that the optimization purpose of the RIO-MY problem is to reduce the railcar dwell time, in this paper, all the above decisions need to be involved to solve the RIO-MY problem. Essentially, it is a highly complex combinatorial optimization problem, which we formulate as a mixed integer nonlinear programming model next.

3.2. Mathematical formulation

The RIO-MY problem contains various decisions with relevant restrictions in practice. To better describe them, we formulate the problem by the mixed integer nonlinear programming (MINLP). According to practical regulations in marshalling yard, some preset assumptions and settings for the MINLP modelling include that: (1) The inbound and outbound trains to be arranged, along with the planning period are specified; (2) The inbound trains arrive at the receiving yard precisely at the scheduled arrival time; (3) The outbound trains will depart from the departure yard precisely at the given departure time unless being cancelled; (4) No disruptions will occur in the humping, disassembly, and assembly processes; (5) Only one hump exists; (6) Two humping and pull-out shunting locomotives are available, respectively; (7) The capacities of the receiving and departure yard are not limited. Table 1 shows notations of sets, parameters, and decision variables in our MINLP model, where the values of parameters and variables are non-negative integers. We detail the constraints and objective in the following content.

3.2.1. Constraints of inbound train humping

In the practical railcar itinerary, each inbound train must be humped and disassembled by a hump shunting locomotive. In addition, the humping should begin after the technical inspection for an inbound train is done. These constraints can be expressed as below,

$$\sum_{l_h \in L_h} \eta_a^{l_h} = 1, \quad \forall a \in A \quad (1)$$

$$\tau_a^{hn} \eta_a^{l_h} + M(1 - \eta_a^{l_h}) \geq u_a^{in} + t_a^{tech}, \quad \forall a \in A, l_h \in L_h \quad (2)$$

where Eq. (1) guarantees only one hump shunting locomotive is assigned to hump an inbound train. Eq. (2) restricts that an inbound train only can be humped after the arrival technical inspection is completed, where M is a large integer number. It is relaxed for a hump shunting locomotive that is not assigned to an inbound train, i.e., $\eta_a^{l_h} = 0$. Humping and disassembly are two continuous processes with no interruptions in between, so the end time of disassembling an inbound train can be calculated as below,

$$\tau_a^{he} = \tau_a^{hn} + t_a^h + \sum_{b \in B_a} t_b^{dis} \quad (3)$$

Table 1

The notations of sets, parameters, and decisions variables.

Set	Description	Variable	Description
A	Set of inbound trains, index by a , i.e., $a \in A$.	$p_{a,a'}$	Hump sequencing binary variable; $p_{a,a'}=1$, if train a is humped before a' ; $p_{a,a'}=0$, otherwise.
D	Set of outbound trains, index by d , i.e., $d \in D$.	$\eta_a^{l_h}$	Hump shunting locomotive scheduling binary variable; $\eta_a^{l_h}=1$, if inbound train a is humped by hump shunting locomotive l_h ; $\eta_a^{l_h}=0$, otherwise.
C	Set of classification tracks, indexed by c , i.e., $c \in C$.	τ_a^{hn}	Start time of humping inbound train a
B	Set of all railcars, indexed by b .	τ_a^{he}	End time of disassembling inbound train a
B_a	$B_a \subset B$, Set of railcars of the inbound train a , index by b_a , i.e., $b_a \in B_a$	τ_a^{le}	End time of all railcars of inbound train a rolls into the classification yard
L_h	Set of hump shunting locomotives, indexed by l_h , i.e., $l_h \in L_h$		
L_s	Set of pull-out shunting locomotives, indexed by l_s , i.e., $l_s \in L_s$		
Parameter	Description		
t_a^{tech}	Technical inspection time after arrival for inbound train a	$q_{d,d'}$	Assembly sequencing binary variable. $q_{d,d'}=1$, if outbound train d is assembled before d' ; $q_{d,d'}=0$, otherwise.
t_d^{tech}	Technical inspection time before departure for outbound train d	$\eta_d^{l_s}$	Pull-out shunting locomotive scheduling binary variable; $=1$, if train d is assembled by pull-out shunting locomotive l_s ; $=0$, otherwise;
u_a^{in}	Scheduled arrival time of inbound train a	$q_{d,d'}^{l_s}$	Assembly sequencing binary variable for the same locomotive; $q_{d,d'}^{l_s}=1$, if train d and d' are both assembled by shunting locomotive l_s and d is assembled before d' ; $q_{d,d'}^{l_s}=0$, otherwise.
u_d^{out}	Scheduled departure time of outbound train d	τ_d^{sn}	Start time of assembling outbound train d
t_a^{in}	Required movement time of pushing over inbound train a to the hump	τ_d^{se}	End time of assembling outbound train d
t_a^{lf}	Required movement time of railcars of inbound train a roll into the classification tracks	x_b^c	Classification track assignment binary variable; $x_b^c=1$, if classification track c is assigned for railcar b to park; $x_b^c=0$, otherwise.
t_h^{sm}	Safety margin time between two consecutive train occupying the hump	y_b^d	Car-to-train assignment binary variable; $y_b^d=1$, if railcar b is assigned to form outbound train d ; $y_b^d=0$, otherwise.
t_s^{lm}	Required movement time of a pull-out shunting locomotive moves to the classification yard	θ_d	Railcar number limit binary variable; $\theta_d=1$, if outbound train d meets the minimum and maximum number limits of railcars; $\theta_d=0$, otherwise.
t_d^{no}	Required time of pulling out and assembling railcars for outbound train d in the classification yard	m_a^d	Inbound and outbound train mapping variable; $m_a^d=1$, if any railcars of inbound train a are assigned to outbound train d ; $m_a^d=0$, otherwise.
t_s^{tr}	Required time of transferring a new outbound train from the classification yard to the departure yard	e_a^d	Classification track real-time capacity calculation variable; $e_a^d=1$, if outbound train d is not cancelled and its assembly end time is before the humping start time of inbound train a ; $e_a^d=0$, otherwise.
λ_d^{min}	Minimum number of railcars for assembling departure train d		
λ_d^{max}	Maximum number of railcars for assembling departure train d		
κ_c	Parking capacity of classification track c		
z_b^d	Binary parameter; $z_b^d=1$, if the destination of railcar b meets the formation requirements of departure train d ; $z_b^d=0$, otherwise.		
g_b^c	Binary penalty parameter; $g_b^c=0$, if the direction of railcar b is identical with the preferred direction of classification track c ; $g_b^c=1$, otherwise.		
a_b	The inbound train that contains railcar b		
u^{end}	The end time of the planning period		

where t_b^{dis} is the duration of disassembling a single railcar. After a safety margin of the hump being occupied, the next inbound train can be humped. Therefore, the constraints of the humping sequence and hump occupation can be formulated as follows,

$$\tau_a^{he} + t_h^{sm} - \tau_{a'}^{hn} \leq M(1 - p_{a,a'}), \quad \forall a, a' \in A \quad (4)$$

$$\tau_{a'}^{he} + t_h^{sm} - \tau_a^{hn} \leq M p_{a,a'}, \quad \forall a, a' \in A \quad (5)$$

where t_h^{sm} is the safety margin of hump, and $p_{a,a'}$ is the hump sequencing variable. If $p_{a,a'}=1$, then inbound train a is humped before inbound train a' . In this case, the end time of disassembling a and the start time of humping a' should be separated by the safety margin, so that Eq. (4) is activated and Eq. (5) is relaxed. On the contrary, if a' is humped before a , Eq. (4) is relaxed and Eq. (5) is activated.

3.2.2. Constraints of train makeup

In practice, the RIO-MY problem involves constraints in terms of the train makeup plan. First, each railcar can only be assigned to at most one outbound train such that,

$$\sum_{d \in D} y_b^d \leq 1, \quad \forall b \in B \quad (6)$$

where y_b^d is the binary assignment variable, meaning railcar b is assigned to outbound train d . If a railcar is not assigned to any outbound train, it keeps parking on the classification track. It should be emphasized that Eq. (6) is applicable to not only railcars disassembled from inbound trains but also railcars that already exist in the classification yard before the planning period of the railcar itinerary. As mentioned in Section 3.1, the train makeup plan should comply with the formulation requirements of outbound trains such that,

$$y_b^d \leq z_b^d, \quad \forall d \in D, b \in B \quad (7)$$

where z_b^d is the given parameter specifying whether railcar b is matched to outbound train d according to the formation requirement of destinations. If $z_b^d = 0$, then y_b^d is forced to equal 0, so that railcar b will never be assigned to outbound train d . In addition, the number of railcars used to form an outbound train is constrained by the minimum and maximum limits, such that

$$\lambda_d^{\min} \theta_d \leq \sum_{b \in B} y_b^d \leq \lambda_d^{\max}, \quad \forall d \in D \quad (8)$$

$$y_b^d \leq \theta_d, \quad \forall d \in D, b \in B \quad (9)$$

where θ_d is the binary variable indicating whether an outbound train satisfies the limits of the railcar number. $\theta_d = 0$ means that before the latest assembly which could lead to the on-time departure of outbound train d , there are no sufficient railcars assigned to this train. In this case, this outbound train should be cancelled, and railcars should not be assigned to it, according to Eq. (9). Otherwise, $\theta_d = 1$, and the outbound train needs to satisfy the limits of the railcar number, as expressed in Eq. (8).

Moreover, the train makeup plan should provide sufficient time for shunting operations. Since the disassembly only refers to decoupling railcars from the inbound trains on the hump, the assembly occurs after the disassembled railcars roll into the classification yard. Accordingly, the time point when all railcars of inbound train a roll into the classification yard can be calculated as below,

$$\tau_a^{le} = \tau_a^{hn} + t_a^h + t_a^{lf}, \quad \forall a \in A \quad (10)$$

where t_a^{lf} signifies the time used for all railcars of inbound train a to roll into the classification yard from the hump. In reality, the specific value of this time depends on the number of railcars and the real-time remaining capacity of classification tracks. Only after the railcars of an inbound train roll into the classification yard, the outbound trains these railcars are assigned to can start to be assembled.

$$\tau_a^{le} \leq \tau_d^{sn} + M(1 - m_a^d), \quad \forall a \in A, d \in D \quad (11)$$

where τ_d^{sn} is the start time of assembling outbound train d . m_a^d is a binary variable that indicates whether any railcars from inbound train a are assigned to the outbound train d . If such an assignment does not exist, $m_a^d = 0$ and Eq. (11) is relaxed. The value of m_a^d is also intertwined with y_b^d , as below,

$$m_a^d \leq \sum_{b \in B_a} y_b^d, \quad \forall a \in A, d \in D \quad (12)$$

$$m_a^d \geq y_b^d, \quad \forall a \in A, d \in D, b \in B_a \quad (13)$$

Eq. (12) forces m_a^d to equal 0 if no railcar from inbound train a is assigned to outbound train d . Eq. (13) forces m_a^d to equal 1 if any railcar from inbound train a is assigned to outbound train d .

3.2.3. Constraints of classification track assignment

In the context of the classification yard, since all inbound trains must be humped, each railcar should have an assigned classification track to park on, such that,

$$\sum_{c \in C} x_b^c = 1, \quad \forall b \in B \quad (14)$$

where x_b^c is a binary variable to indicate the classification track assignment for railcars. As mentioned in Section 3.1, the first-in-first-out principle is applied to the roll-in and pull-out of railcars on a classification track. The sequence of railcars entering and exiting a classification track is related to the humping sequence and assembly sequence, which are expressed as follows,

$$\sum_{d \in D} \tau_d^{sn} y_b^d - \sum_{d' \in D} \tau_{d'}^{sn} y_{b'}^{d'} \leq M(3 - x_b^c - x_{b'}^c - p_{a_b, a_{b'}}), \quad \forall c \in C, b, b' \in B \quad (15)$$

$$\sum_{d' \in D} \tau_{d'}^{sn} y_{b'}^{d'} - \sum_{d \in D} \tau_d^{sn} y_b^d \leq M(2 - x_b^c - x_{b'}^c + p_{a_b, a_{b'}}), \quad \forall c \in C, b, b' \in B \quad (16)$$

where $\sum_{d \in D} \tau_d^{sn} y_b^d$ and $\sum_{d' \in D} \tau_{d'}^{sn} y_{b'}^{d'}$ are the start time of assembling the two outbound trains that railcar b and b' are assigned to, respectively. Eqs. (15) and (16) are activated when both x_b^c and $x_{b'}^c$ equal to 1, meaning that railcars b and b' are both assigned

to classification track c . a_b and $a_{b'}$ signify inbound trains that contain railcar b and b' , respectively. If a_b is humped before $a_{b'}$, i.e., $p_{a_b, a_{b'}} = 1$, Eq. (16) is relaxed and Eq. (15) is converted to $\sum_{d \in D} \tau_d^{sn} y_b^d \leq \sum_{d' \in D} \tau_{d'}^{sn} y_{b'}^{d'}$, meaning that the outbound train with the assigned railcar b is assembled before the train with the assigned railcar b' . Consequently, railcar b enters classification track c before railcar b' , and also leaves the track before railcar b' . Hence the first-in-first-out principle is guaranteed. Similarly, Eq. (15) is relaxed and Eq. (16) is hold if a_b is humped after $a_{b'}$, i.e., $p_{a_b, a_{b'}} = 0$.

On the other hand, the capacity of any classification track is constrained as below,

$$\sum_{a' \in A} \sum_{b \in B_{a'}} x_b^c p_{a', a} + \sum_{b \in B_a} x_b^c - \sum_{d \in D} \sum_{b \in B} x_b^c y_b^d \epsilon_a^d \leq \kappa_c, \quad \forall a \in A, c \in C \quad (17)$$

Eq. (17) guarantees the sufficient available capacity on the assigned classification tracks for inbound train a . The first term on the left side means the total number of railcars that are from inbound trains humped before inbound train a and assigned to classification track c . The second term means the number of railcars assigned to classification track c from inbound train a . The third term means the total number of railcars that are pulled out from track c before inbound train a is humped. Therefore, the left side of Eq. (17) is the total number of railcars on classification track c after all railcars of inbound train a roll into classification tracks. This number cannot exceed the maximum capacity of classification track c , i.e., κ_c . It is noteworthy that the third term is related to the binary variable ϵ_a^d , which indicates whether outbound train d is already assembled before inbound train a is humped. Accordingly, the variable ϵ_a^d is determined by the start time of humping inbound train a and the end time of assembling outbound train d , such that,

$$\tau_a^{se} - \tau_a^{hn} \leq M(1 - \epsilon_a^d), \quad \forall a \in A, d \in D \quad (18)$$

$$\tau_a^{hn} - \tau_a^{se} \leq M\epsilon_a^d, \quad \forall a \in A, d \in D \quad (19)$$

3.2.4. Constraints of outbound train assembly

Regarding the outbound train assembly, the start and end time of assembling an outbound train, i.e. τ_d^{sn} and τ_d^{se} , are expressed as follows,

$$\tau_d^{se} = \tau_d^{sn} + (t_s^{lm} + t_d^{po} + t_s^{tr})\theta_d, \quad \forall d \in D \quad (20)$$

$$\tau_d^{sn} \leq M\theta_d, \quad \forall d \in D \quad (21)$$

$$(\tau_d^{se} + t_d^{tech})\theta_d \leq u_d^{out}, \quad \forall d \in D \quad (22)$$

where t_s^{lm} , t_d^{po} and t_s^{tr} signify the required time of a pull-out locomotive moving to the classification yard, assembling railcars, and transferring new outbound trains to the departure yard, respectively. In real-world operations, for each new outbound train, the specific values of the above times are determined by the number and location of classification tracks, because the pull-out shunting locomotive must enter and exit the classification yard multiple times to collect railcars parking on different classification tracks. It should be noted that, if an outbound train is cancelled, θ_d in Eqs. (20) and (21) equal to 0 and force the start and end time of assembly to be 0, which will not conflict with the assembly of other outbound trains. Eq. (22) ensures that after the assembly, each outbound train has enough time to complete the technical inspection before the scheduled departure time. On the other hand, the assembly of each outbound train needs to be done by a pull-out shunting locomotive, which is expressed as below,

$$\sum_{l_s \in L_s} \eta_d^{l_s} = \theta_d, \quad \forall d \in D \quad (23)$$

where $\eta_d^{l_s}$ is the binary variable for scheduling pull-out shunting locomotives to assemble outbound trains. If an outbound train is cancelled, i.e., $\theta_d = 0$, no pull-out shunting locomotive needs to be scheduled for it. If any two outbound trains b and b' are assembled by the same pull-out shunting locomotive, the two assembly tasks should satisfy the following temporal constraints,

$$\tau_d^{se} \eta_d^{l_s} - \tau_{d'}^{sn} \eta_{d'}^{l_s} \leq M(1 - q_{d, d'}^{l_s}) + M(2 - \eta_d^{l_s} - \eta_{d'}^{l_s}), \quad \forall d, d' \in D, l_s \in L_s \quad (24)$$

$$\tau_{d'}^{se} \eta_{d'}^{l_s} - \tau_d^{sn} \eta_d^{l_s} \leq Mq_{d, d'}^{l_s} + M(2 - \eta_d^{l_s} - \eta_{d'}^{l_s}), \quad \forall d, d' \in D, l_s \in L_s \quad (25)$$

Eqs. (24) and (25) are activated if $\eta_d^{l_s}$ and $\eta_{d'}^{l_s}$ both equal to 1, which means outbound train d and d' are assembled by pull-out shunting locomotive l_s . If d is assembled before d' , i.e., $q_{d, d'}^{l_s} = 1$, Eq. (24) is converted to $\tau_d^{se} \eta_d^{l_s} \leq \tau_{d'}^{sn} \eta_{d'}^{l_s}$, which indicates the end time of assembling outbound train d is before the start time of assembling outbound train d' . Therefore, the assemblies can be completed successively without conflicts. Otherwise, i.e., $q_{d, d'}^{l_s} = 0$, Eq. (24) is relaxed and Eq. (25) guarantees the conflict-free assembly operations.

3.2.5. Model formulation

The objective of the RIO-MY problem is to reduce the railcar dwell time, so as to raise the operational efficiency of the marshalling yard. As mentioned in Section 3.1, the prioritized classification tracks with preferred destinations are given priority to be assigned to railcars with identical destinations. To this end, a penalty item is set in the objective function for reducing the chance that non-prioritized classification tracks are assigned to railcars, which is expressed as below,

$$\min. \sum_{b \in B} \sum_{d \in D} (u_d^{out} - u_{a_b}^{in}) y_b^d + \sum_{b \in B} \sum_{c \in C} g_b^c y_b^c \quad (26)$$

where g_b^c is the given binary penalty parameter. If a non-prioritized classification track c is assigned to railcar b , a penalty $g_b^c = 1$ will be added to the objective function, according to the second term in Eq. (26). In doing so, the prioritized classification tracks will be considered first in the assignment. The first term in Eq. (26) means the railcar dwell time in the marshalling yard, which is derived from the arrival times of inbound trains (which comprise the railcars) and departure times of outbound trains (which the railcars are assigned to). If a railcar is not assigned to any outbound train, the end time of the planning period t^{end} is used as the departure time to calculate the railcar dwell time. Therefore, the above objective function is able to reduce railcar dwell time and simultaneously incline towards the priority of classification track assignment. Given the above constraints and objective, we formulate the RIO-MY problem as the following MINLP model,

Minimization: Both the railcar dwell time and the penalty: Eq. (26)

Subject to: Constraints of inbound train humping: Eqs. (1) to (5);

Constraints of train makeup: Eqs. (6) to (13);

Constraints of classification track assignment: Eqs. (14) to (19);

Constraints of outbound train assembly: Eqs. (20) to (25).

The MINLP model can be solved by some mature mathematical programming solvers, but they still suffer from a considerable computational challenge because of the discrete and non-linear nature. In particular, the number of integer variables could be easily up to millions for the problem instances with large sizes. In addition, the parameters in the MINLP model are fixed, leading to the neglect of dynamics, such as the changing time of railcars rolling into tracks (t_a^{lf}) and outbound train being assembled (t_d^{po}), which are affected by the state of the marshalling yard, e.g., the distribution of railcars on the classification tracks. The missing representation of real-world dynamics may induce inferior solutions to the RIO-MY problem. Some heuristic algorithms have been customized for similar problems, which are not the exact RIO-MY problem in this paper. Our RIO-MY problem involves more practical constraints of train shunting operations, especially the capacity and preferred destination of classification tracks. Meanwhile, the existing heuristic algorithms rely on massive labor to design hand-crafted rules and tune algorithmic configurations, which hinders the exploration of high-quality solutions. Consequently, the improvement of computational efficiency and performance is still in great demand. In this paper, we attempt to apply the DRL method to solve the RIO-MY problem. The DRL method saves much labor to specialize and tune an algorithm, and the learned policy can deliver high-quality solutions efficiently. Meanwhile, the DRL method is able to handle real-time changing parameters in the dynamics of railcar itinerary. Despite learning policies for all decisions, our DRL method is focused on crucial decisions on two specific shunting operations, and leaves the other decisions by heuristics. We introduce such a hybrid mode in the next section.

3.3. DRL for crucial decisions in RIO-MY

Although DRL methods are good at automatically learning policies, developing end-to-end models to tackle all decisions in a complex combinatorial optimization problem is not an easy job. Especially for the RIO-MY problem, directly learning various decisions stated in the above Section 3.2 could induce a huge action space in DRL and thus an overly high complexity in DRL. To avoid such issues, we concentrate the strength of DRL on crucial decisions for hump sequencing and classification track assignment, and leave other decisions (e.g., for outbound train assembly and shunting locomotive scheduling) solved by heuristic rules which are integrated into the environment of DRL. The decisions for hump sequencing and classification track assignment are crucial, as they largely determine the dynamic railcar availability on classification tracks for subsequent outbound train assembly (Jaehn et al., 2015b; Haahr and Lusby, 2016). The railcar availability further leads to significant effects on the railcar itinerary and thus the railcar dwell time. Especially, if a non-prioritized classification track is assigned to a railcar, railcars of different destinations on this mixed-used track could induce obstructions among railcars in assembly due to the FIFO principle. Given the real-time railcar availability determined by the crucial decisions by the DRL agent, we use a heuristic rule for tackling the outbound train assembly, which simultaneously determines the sequence and formation for assembling outbound trains with the purpose of reducing railcar dwell time. Moreover, the heuristic rule for the shunting locomotive scheduling is also integrated into the environment to coordinate with the disassembly and assembly. The results of outbound train assembly by the above heuristic rules are considered in the reward definition for DRL. In doing so, we involve the effect of the heuristic rules in policy learning to yield more favourable DRL decisions for hump sequencing and classification track assignment.

The proposed hybrid DRL method for the RIO-MY problem, including both learning and non-learning parts, is different from conventional optimization methods. In this manner, we relieve the complexity of optimizing entire decisions in the RIO-MY problem solely by DRL methods. On the other hand, we would like to note that the train makeup plan is finalized according to the joint results of all decisions (i.e., learned or heuristic decisions) for train shunting operations, and all train operation constraints listed in Section 3.2 are integrated in the environment of DRL. Therefore, the complex interdependency among different train shunting operations and the train makeup plan is implicitly explored by training the DRL decisions, and the feasibility of the integrated train operation plan is always guaranteed.

4. Solving method

This section details the proposed GNN based DRL method for the RIO-MY problem, including the entire framework, the MDP modelling, the state encoding, the policy network, and the training algorithm.

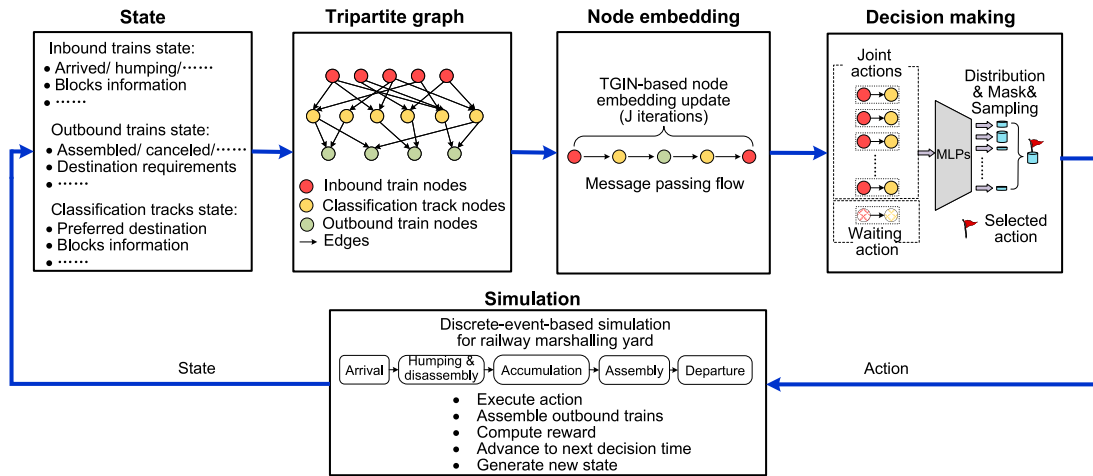


Fig. 2. The framework of GNN based DRL method for the RIO-MY problem.

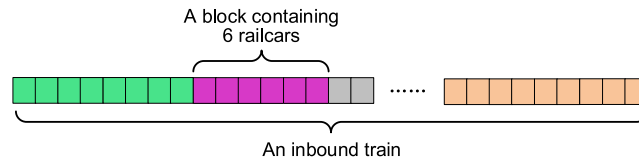


Fig. 3. An illustration of blocks with colours meaning destinations of railcars.

4.1. Framework

This paper proposes a novel GNN based DRL method for the RIO-MY problem. Given the sequential decision-making nature, we model the solving process of RIO-MY problem by an MDP, where the agent iteratively takes joint actions to complete train shunting operations until the terminal state of the planning period. In each iteration, as shown in Fig. 2, the current state reflecting statuses of trains and tracks is first obtained from the environment of DRL (i.e., the discrete-event simulation model of the railway marshalling yard). Then, we encode the state into a tripartite graph containing nodes of inbound trains, outbound trains, and classification tracks and directed edges among nodes. After that, a novel tripartite graph isomorphism network (TGIN) is applied on the graph encoded state to perform message-passing and aggregation for extracting informative embeddings of nodes. Based on the embeddings, the decision-making network (DMN) is used to compute the probability distribution over actions (i.e., joint actions to decide on both hump sequencing and classification track assignment), from which an action is sampled for the current state. Finally, the selected action is executed in the environment of the railway marshalling yard to facilitate the transition, obtain the reward, and advance to the next state. The TGIN and DMN constitute the entire policy network for the RIO-MY problem, which is trained by the proximal policy optimization (PPO) algorithm with the collected trajectories, i.e., sequences of states, actions, and rewards.

4.2. MDP formulation

4.2.1. The block

To implement the DRL method, the solving process of our RIO-MY problem needs to be formulated as a Markov decision process (MDP). It should be noted that, in practice, the disassembly and assembly take the **block** rather than railcar as the operation unit. Hence, block-wise decisions in MDP enable it to be more close to the practical operations. Specifically, a **block** is a group of railcars that are merged in a head-to-tail manner. The railcars in each block are homogeneous with the same destination. Thus, a block is also associated with priority classification tracks, which are the same as the priority classification tracks of railcars in the block.

The relationship between blocks and railcars in a train is illustrated in Fig. 3. In real-world regulation, an inbound train consists of multiple blocks, and blocks are later assembled to form an outbound train. To approximate this reality, we define that the agent in MDP executes block-wise decisions at each step. On the other hand, since the number of blocks is much less than railcars in a train, the block-wise decision is advantageous in maintaining the length of MDP episodes within a reasonable range. In contrast, the railcar-wise decisions lead to long episodes with thousands of transitions, which could suffer from issues like large state space, delayed reward, and low training efficiency. Last but not least, we allow the agent to disassemble a part of a block from an inbound train. Such a decomposition is implemented if the current block cannot park on the assigned classification track as a whole due to insufficient track capacity. In doing so, we inject flexibility into the block-wise decisions to make full use of the capacity of classification tracks. In the following sections, we detail key components of the MDP.

Table 2

Features of inbound trains, outbound trains, and classification tracks in a state.

Entity	Features (Static and dynamic features begin with · and ★, respectively.)
Inbound train	★ Status (not arrive/ not humped/ humped); arrival index; · formation (destinations of blocks); ★ humping index; ★ humping start time; ★ index of block being disassembled; ★ real-time number of railcars
Classification track	· Preferred destination; ★ current railcars parking on it; ★ current railcars belong to which inbound trains
Outbound train	★ Status (not assembled/ assembled/ cancelled); · formation requirement (destinations); ★ the start and end time of assembly; ★ the determined formation

4.2.2. State

In our MDP for the RIO-MY problem, the state S_i , at i th step of a decision episode, is observed by the agent at the decision moment of disassembling a block, i.e., when a block needs to be disassembled from an inbound train and a classification track needs to be assigned to it simultaneously. Specially, during the disassembly process of an inbound train, when a block needs to be disassembled from the inbound train on the hump, the agent should decide on which classification track is assigned to the block for parking. At this moment, recorded as i th step, the state S_i are generated for the agent to reason out an action (i.e., a decision). After the block is disassembled and sent to the assigned track according to the agent's action, the next block needs to be disassembled from the inbound train, which means the next decision moment (i.e., $(i + 1)$ th step) with the state S_{i+1} . After all blocks composing a single inbound train are completely disassembled, the agent should decide on which inbound train to be humped next for disassembling its first block and which classification track should be assigned to this block. Accordingly, the episode of our MDP consists of steps defined by the decision moments.

In each step, a state S_i should reflect both the static and dynamic information in the marshalling yard, which affect the decisions on the hump sequencing and classification track assignment. We choose some simple but key features to represent the two aspects, e.g., the destinations of blocks in inbound and outbound trains as static features, the start time of humping and assembly as dynamic features, etc. The full list of features we use in this paper is displayed in Table 2, and their detailed descriptions are presented in Appendix B. As mentioned in Section 3, there exists complex interdependency among shunting operations such as hump sequencing, classification track assignment, and assembly sequencing. In this paper, we leverage GNN to learn informative state representations for decision-making, by which the agent could be more aware of the interplay of different operations in the marshalling yard. To this end, we encode the state with all features in Table 2 into a tripartite graph that is take as input to GNN. The graph structure and GNN are introduced in Sections 4.3 and 4.4, respectively.

4.2.3. Action

In this paper, we focus on applying DRL to automatically learn decisions on the hump sequencing and classification track assignment. Hence, in the MDP formulation, we define the *joint action* executed by the agent under state S_i as $\delta_i = (a, c)$. The corresponding decision is that the inbound train a is selected to disassemble a block from it and a classification track c is assigned to the block for parking. Given the fact that an inbound train is disassembled block by block, the train a in the joint action elegantly signifies one exact block to be disassembled under the current state. In comparison to directly selecting a block, selecting an inbound train can better unify the joint actions under different states, and avoid a huge action space since the number and type of blocks are much more than those of trains.

Joint actions during disassembling an inbound train. Specifically, given the currently being disassembled inbound train a and its block β_a to be disassembled under the current state S_i , the available actions under this state (i.e., Δ_{S_i}) only contain the available joint actions for the inbound train a (i.e., Δ_a). Hence, the set of available actions is expressed as below,

$$\Delta_{S_i} = \Delta_a = \{(a, c) | \forall c \in C_{S_i}^{\beta_a}\} \quad C_{S_i}^{\beta_a} \subset C \quad (27)$$

where $C_{S_i}^{\beta_a}$ denotes the set of classification tracks that can be assigned to block β_a under state S_i . As mentioned in Section 3.1, the prioritized classification tracks of the block β_a are first considered as $C_{S_i}^{\beta_a}$. Only if all prioritized classification tracks have no sufficient capacity, the non-prioritized ones are set as $C_{S_i}^{\beta_a}$. Formally, we describe the available classification tracks for block β_a under state S_i as below,

$$C_{S_i}^{\beta_a} = \begin{cases} \{c | Cap_{S_i}^c > 0, \forall c \in C_{pr}^{\beta_a}\} & \text{if } \sum_{\forall c \in C_{pr}^{\beta_a}} Cap_{S_i}^c > 0 \\ \{c | Cap_{S_i}^c \geq k_{\beta_a}, \forall c \in C_{np}^{\beta_a}\} & \text{if } \sum_{\forall c \in C_{pr}^{\beta_a}} Cap_{S_i}^c = 0 \end{cases} \quad (28)$$

where $Cap_{S_i}^c$ denotes the remaining capacity on classification track c under state S_i ; $C_{pr}^{\beta_a}$ and $C_{np}^{\beta_a}$ denote the sets of prioritized and non-prioritized classification tracks with respect to block β_a , respectively; k_{β_a} denotes the number of railcars in block β_a . If any prioritized classification track still has remaining capacity, it should be included in $C_{S_i}^{\beta_a}$ as an optional assignment to block β_a , and meanwhile all non-prioritized classification tracks should be excluded. In this case, the remaining capacity of a prioritized classification track may not be sufficient for the entire block to park. To handle the capacity insufficiency, the block decomposition is used as shown in Fig. 4. In specific, the block is divided into two sub-blocks. Regarding one sub-block, the number of railcars

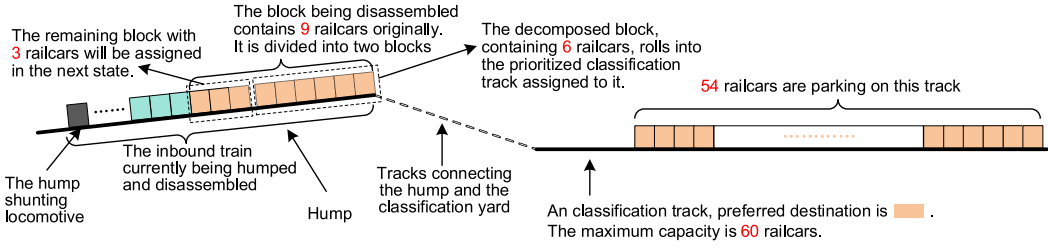


Fig. 4. An example of block decomposition.

is identical to the remaining capacity of the assigned classification track, and thus this sub-block can park on the track. Regarding the other sub-block, it will be assigned to another track in the next state. Such block decomposition can provide more flexibility to make full use of the capacity of the classification tracks. On the other hand, if all prioritized classification tracks are totally occupied, non-prioritized classification tracks with sufficient remaining capacity for block β_a are considered, as expressed by Eq. (28). The block decomposition is not allowed for non-prioritized classification tracks, in order to avoid too much messy distribution of railcars in the classification yard.

Joint actions launching a new disassembly task. The above Eqs. (27) and (28) formulate the classification track assignment during the disassembly of a specific inbound train a . In other words, during the continuous multiple steps of disassembling an inbound train, the available action sets for each state only contain actions related to blocks from this inbound train. In contrast, if an inbound train has been completely disassembled, the agent under the current state S_i should select both the next inbound train to be humped and the classification track for the first block in the train to park. In this situation, all inbound trains that are currently in the receiving yard and ready to be humped should be included in the available joint actions, which are formally expressed below,

$$\Delta_{S_i}^{joint} = \left\{ \bigcup_{\forall a \in \hat{A}_{S_i}} \Delta_a \right\} \quad (29)$$

where $\hat{A}_{S_i}$ denotes the set of inbound trains that are ready to be humped under state S_i ; Δ_a denotes the set of available joint actions regarding the first block in the inbound train a , as expressed by Eqs. (27) and (28). In other words, $\Delta_{S_i}^{joint}$ represents all available tracks for the first blocks of inbound trains that are ready to be humped for disassembly. Therefore, by selecting a joint action from $\Delta_{S_i}^{joint}$, the agent is able to conduct hump sequencing and launch a new inbound train to be disassembled, since it determines the next inbound train to be humped and the classification track assigned to the first block in the train, simultaneously.

Waiting action. Furthermore, we design a waiting action ω , which means the agent is allowed to abandon all inbound trains that are ready to be humped but select to disassemble an inbound train that arrives later. Accordingly, the available action space under this state is extended as below,

$$\Delta_{S_i} = \Delta_{S_i}^{joint} \cup \{\omega\} \quad (30)$$

By the waiting action, more inbound trains can be considered in the decisions on which train to be humped next, including not only the ones that are ready for disassembly but also the ones that have not yet arrived at the current state. Additionally, if the waiting action is selected under state S_i , the state transits to the next state S_{i+1} , which is at the moment that the next inbound train $\bar{a}$ finishes its arrival technical inspection. To explore more possibilities on humping sequence, the available action space under S_{i+1} is masked and can be expressed by,

$$\Delta_{S_{i+1}} = \Delta_{\bar{a}} \cup \{\omega\} \quad (31)$$

where $\Delta_{\bar{a}}$ is the available joint actions of inbound train $\bar{a}$ (as Eq. (27)). In other words, Eq. (31) suggests that once a waiting action is selected under a state, the action space in the succeeding state comprises only the joint actions regarding the next arriving inbound train and the waiting action. We note that the trains ready to hump in the receiving yard are excluded in the available action space under state S_{i+1} . The reason is selecting the waiting action under state S_i means trains in $\hat{A}_{S_i}$ will not be the next ones to be humped. If trains in $\hat{A}_{S_i}$ are included in the available action space under state S_{i+1} , they could be selected to be the next humped train again, so that the humping sequence might not be changed after the waiting action under state S_i , violating the purpose of the waiting action and impairing the exploration efficiency.

In summary, the available action space Δ_{S_i} varies along with states. An action reflects the integrated decisions on both hump sequencing and classification track assignment. After an inbound train is completely disassembled, the agent selects actions from Eqs. (30) and (31) to determine the next train to be disassembled and the classification track assigned to its first block. During the continuous disassembly process of a specific inbound train, the agent selects actions according to Eq. (27), in order to sequentially determine the classification track assignment. The above action masking stimulates the agent to explore meaningful actions during the training processes to effectively learn a policy aiming to minimize railcar dwell time. Consequently, a trained agent is expected to reasonably decide which train to be humped or waiting for the next inbound train.

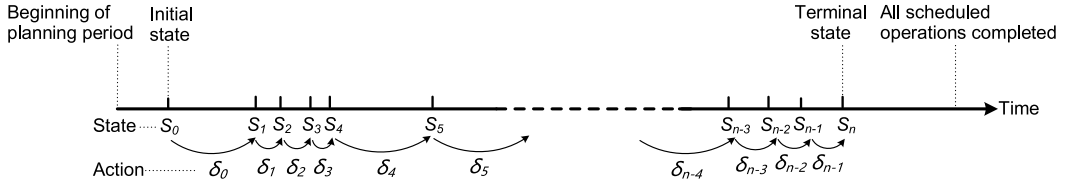


Fig. 5. Transitions in the MDP.

4.2.4. Transition

The transitions in our MDP are depicted in Fig. 5. Given the train timetable, the initial state S_0 is defined as either the moment when the first inbound train finishes the arrival technical inspection, or the beginning of the planning period if any inbound train already exists in the receiving yard. For each state S_i at i th step, the available action space is established according to Eqs. (27) to (31), as explained in Section 4.2.3. The agent selects an action δ_i , which instigates that the state S_i transits to the next state S_{i+1} . It should be emphasized that the transition time between two adjacent states is not fixed, which is mainly dependent on the time to execute the selected action. Specifically, according to Sections 4.2.2 and 4.2.3, the transition time is equivalent to the time used to disassemble a block from the inbound train after a joint action is selected by the DRL agent. However, if the joint action is executed to disassemble the last block of an inbound train or the waiting action is selected, the time to finish the arrival technical inspection for the next inbound train should be involved in the transition time. In parallel with the above operations guided by the agent, the other shunting operations proceed continuously in transitions. For example, the blocks roll into classification tracks, the hump/pull-out shunting locomotives are scheduled for the next disassembly/assembly, and outbound trains are assembled using blocks from classification yards. Correspondingly, the features of the state, as shown in Table 2, are varying with transitions. The terminal state is defined as the moment that all inbound trains in the given timetable are disassembled, which means the end of an episode of MDP. In this state, the assembly and departure operations continue in the marshalling yard until all outbound trains finally depart.

Notably, the aforementioned shunting operations that are not guided by the agent during transitions also need decision making. Since we focus on learning decisions on hump sequencing and classification track assignment, the outbound train assembly and shunting locomotive scheduling are implemented by two practical heuristic rules in the environment of DRL. The heuristic rule for assembling outbound trains is used to determine the assembly sequence and formation of each outbound train, with the aim of reducing railcar dwell time. In specific, after the action δ_i is executed under state S_i , the heuristic rule attempts to assemble an outbound train by checking the following practical constraints: (1) whether blocks are available to be pulled out and satisfy the formation requirement; (2) whether the number of railcars in such qualified blocks meets the minimum/maximum railcar number limit; (3) whether the outbound train can depart on time after shunting operations of assembly. In process (2), the qualified blocks are counted in the ascending order of their arrival time, and assigned to the outbound train in process (3). This heuristic rule also cancels an outbound train if the minimum railcar number limit is not reached before the deadline of its on-time departure. Accordingly, the assembly sequence and formation of outbound trains are determined according to the real-time railcar availability and doable train shunting operations. In this manner, the train makeup plan can be determined by train shunting operations during the transitions of MDP. We detail this heuristic rule by Algorithm A.1 in Appendix A. On the other hand, the heuristic rule for shunting locomotive scheduling simply allocates the earliest available locomotive to serve the next task. It determines the start time to hump an inbound train and assemble an outbound train, respectively.

4.2.5. Reward

In a typical MDP, after the agent selects an action under a state, it leads to the state transition in the environment which provides a reward as feedback to the agent. However, the objective of the RIO-MY problem is to reduce the average railcar dwell time, which can only be obtained at the terminal state of the episode. To bypass this delayed feedback, we propose a reward shaping scheme to specify the rewards for each state. It benefits the policy learning in DRL by the immediate rewards, so as to render the agent learn better decisions towards reducing the ultimate railcar dwell time. In specific, we propose to compute an estimated average railcar dwell time $T(S_i)$ for a state S_i as below,

$$T(S_i) = \frac{\sum_{b \in B} t_b^i}{|B|}, \quad t_b^i = \begin{cases} u_d^{\text{out}} - u_{ab}^{\text{in}} & \text{railcar } b \text{ is determined to form outbound train } d \\ u_{d'}^{\text{out}} - u_{ab}^{\text{in}} & \text{railcar } b \text{ is estimated to form outbound train } d' \\ u^{\text{end}} - u_{ab}^{\text{in}} & \text{railcar } b \text{ has not been disassembled} \end{cases} \quad (32)$$

where t_b^i denotes the dwell time of railcar b in state S_i and B denotes the set of all railcars. As displayed in Eq. (32), the dwell time t_b^i is computed according to three categories of railcars. (1) **Determined railcar**: The railcar b has already been determined to form the outbound train d . Accordingly, the dwell time is computed as per the determined assignment, i.e., the scheduled departure time of the outbound train d minus the arrival time of the inbound train that contains railcar b . (2) **Pending railcar**: The railcar b has been disassembled and is parking in the classification yard, but it has not been assigned to an outbound train. In this situation, an outbound train d' is chosen to be the estimated assignment for the railcar b , and then t_b^i is computed in the same way as done for

the determined railcar. (3) **Succeeding railcar**: The railcar b has not been disassembled, e.g., has not arrived or has arrived but not been disassembled. In this situation, the railcar is temporarily impossible to be assigned to outbound trains. Therefore, the end time of the planning period (i.e., u^{end}) is used to calculate t_b^i .

Estimated assignment. Regarding a pending railcar, an outbound train needs to be chosen as the estimated assignment to the railcar for the computation of t_b^i . Except for the outbound trains that have been assembled and cancelled, We first choose valid ones from the remaining outbound trains, by simply checking if their formation requirements contain the destination of railcar b . Then, from the valid ones, we choose the outbound train with the earliest or latest scheduled departure time. These two choices yield different reward distributions over states, which are tested and compared in our experiments. Please note that if there is no valid outbound train found for a railcar, we regard it as a succeeding railcar to compute the dwell time.

Railcar category conversion. The category of a railcar is converted during transitions. In specific, if an action under a state is not the waiting action, a block is disassembled from an inbound train. Hence the railcars in this block are converted from *succeeding railcars* to *pending railcars*. If an outbound train is determined to be assembled after an action is executed, all railcars assigned to this outbound train are converted from *pending railcars* to *determined railcars*. At the terminal state, all railcars that are not assigned to any outbound train are converted back to *succeeding railcars*, so that the final railcar dwell time is obtained.

Reward definition. Based on the above transformation and the average estimated railcar dwell time $T(S_i)$, the reward at each step of MDP can be defined as $R(S_i, \delta_i, S_{i+1}) = T(S_i) - T(S_{i+1})$, i.e., the difference between the average estimated railcar dwell time under two adjacent states. By setting the discount factor $\gamma = 1$, the cumulative reward for an episode is expressed as below,

$$\sum R(S_i, \delta_i, S_{i+1}) = T(S_0) - T(S_n) \quad (33)$$

where $T(S_0)$ and $T(S_n)$ are the average estimated railcar dwell time at the initial and terminal state, respectively. For a specific RIO-MY problem instance, $T(S_0)$ is a fixed value, since no action has been executed and all railcars in inbound trains are succeeding railcars. Therefore, maximizing Eq. (33) is equivalent to minimizing $T(S_n)$, i.e., minimizing the average railcar dwell time.

We note that the reward function does not completely incorporated with the objective function (Eq. (26)) of the MINLP model. The penalty for using non-prioritized classification tracks in Eq. (26) is not modelled in the reward function. The reason is that the penalty in the MINLP model is a soft constraint to entail the consideration of following the rule of using prioritized classification tracks. However, in our DRL method, the rule of using prioritized classification tracks is already achieved by the masked action space, as we stated in Section 4.2.3. Therefore, the penalty is not necessary in the reward function. Meanwhile, since the inefficient hump sequencing and classification track assignment provoke longer average railcar dwell time, the actions inducing such inefficiency are expected to be automatically suppressed via DRL by using our reward function.

4.2.6. Environment

In the MDP, the agent sequentially selects actions to interact with the environment, which involves the action execution, state transition, and reward feedback. In this paper, the environment is modelled by a discrete-event simulation of the railway marshalling yard. We develop the simulation model using the SimPy package in Python, and present its details in Appendix C. In a nutshell, we implement the heuristic rules for outbound train assembly and shunting locomotive scheduling and reward computation in state transitions. Other necessary train operations, such as the railcar movement and the arrival/departure technical inspection, are also implemented. Meanwhile, the train operation constraints in Section 3.2 are satisfied in the simulation model to guarantee the regulations or restrictions in the train traffic management. Last but not least, some conditions to terminate the MDP episode are set according to the track availability. If a block is disassembled but there is no available classification track for it, the episode is terminated. If an outbound train is determined to be assembled but no berthing track in the departure yard is available, or an inbound train arrives but no berthing track in the receiving yard is available, the episode is terminated.

4.2.7. Policy

By the MDP modelling, we aim to learn a stochastic policy π that is adopted to yield a conditional probability distribution over all possible actions given a state. In doing so, the sequential decision-making for hump sequencing and classification track assignment can be represented by the probability chain, i.e., $\mathcal{P}(S_n|S_0) = \prod_{i=0}^{n-1} \mathcal{T}(S_{i+1}|\delta_i, S_i)\pi(\delta_i|S_i)$. Given the transitions denoted by $\mathcal{T}$ in the environment, we apply the DRL algorithm to learn a policy that the agent follows to make decisions. To this end, we parameterize the policy π by a policy network π_θ , in which a tripartite graph isomorphism network (TGIN) and a decision-making network (DMN) are trained to reason out actions given the graph encoded states. According to the designed rewards in Section 4.2.5, the policy network π_θ can be optimized towards minimizing the railcar dwell time during training. In the following sections, we elucidate the tripartite graph based state encoding, the TGIN based novel policy network, and the training algorithm.

4.3. State encoding by tripartite graph

A state in the process of solving the RIO-MY problem is characterized by the features of inbound trains, classification tracks, and outbound trains, as displayed in Table 2. To better represent the interactions between entities in the marshalling yard, we encode the state as a tripartite graph, i.e., a type of graph structured by three types of nodes and edges only linking nodes in different types.

Specifically, the state S_i is encoded as a tripartite graph $G_i = \{\mathcal{V}_a, \mathcal{V}_c, \mathcal{V}_d, \mathcal{L}_{a,c}^i, \mathcal{L}_{c,d}^i\}$, as shown in Fig. 6(a). $\mathcal{V}_a$, $\mathcal{V}_c$, $\mathcal{V}_d$ denote the sets of inbound train nodes, classification track nodes, and outbound train nodes, respectively. The node in each set represents an individual train or track. Hence, the number of nodes in each type is equivalent to the number of inbound trains, classification

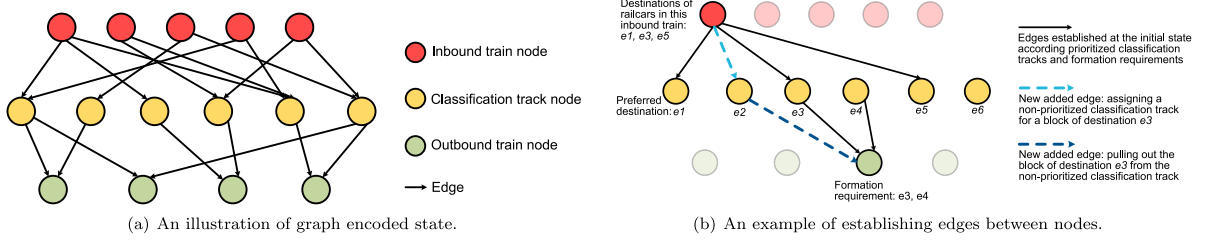


Fig. 6. The tripartite graph based state encoding.

tracks, and outbound trains, respectively, which remain constants in all states. On the other hand, the set of directed edges $\mathcal{L}_{a,c}^i$ go from inbound train nodes to classification track nodes, and the set of directed edges $\mathcal{L}_{c,d}^i$ go from classification track nodes to outbound train nodes. Both $\mathcal{L}_{a,c}^i$ and $\mathcal{L}_{c,d}^i$ alter with the state transitions, and the initial $\mathcal{L}_{a,c}^0$ and $\mathcal{L}_{c,d}^0$ are created under state S_0 . Specifically, as the black edges shown in Fig. 6(b), if a classification track is the prioritized track for any railcar in an inbound train, the classification track node and inbound train node are connected by a directed edge. Similarly, if the preferred destination of a classification track is the same as any required destination of the outbound train, the classification track node and outbound train node are connected by a directed edge. Given the node $a \in \mathcal{V}_a$, $c \in \mathcal{V}_c$ and $d \in \mathcal{V}_d$, we define their raw feature vectors as $v_a \in \mathbb{R}^{66}$, $v_c \in \mathbb{R}^{121}$ and $v_d \in \mathbb{R}^{68}$, respectively, which gather the features of the inbound train, classification track and outbound train in Table 2. Given the edge $(a \rightarrow c) \in \mathcal{L}_{a,c}^i$ and $(c \rightarrow d) \in \mathcal{L}_{c,d}^i$, their raw feature vectors are defined as $l_{a,c} \in \mathbb{R}^2$ and $l_{c,d} \in \mathbb{R}^2$, respectively. Details on the raw features of the tripartite graph are provided in Appendix B.

Dynamics. While the node sets in the tripartite graph remain unchanged in different states, the edge sets together with the raw feature vectors of nodes and edges dynamically change during state transitions. Some examples are as follows. Regarding the features, if an inbound train begins to be humped in a state transition, its status will change to ‘humped’; if a block is disassembled, the real-time number of railcars in the inbound train and the real-time blocks parking on the assigned classification track will be updated; once an outbound train is determined to be assembled, its actual formation and the start and end time of assembly are determined; and so on. Regarding the edge sets, new edges would be added in some state transitions, as the dashed edges shown in Fig. 6(b). Specifically, if an action assigns a non-prioritized classification track to a block disassembled from an inbound train, a new edge connecting the inbound train node and the classification track node will be added to the tripartite graph. Similarly, if a block is assigned to an outbound train and is pulled out from the classification track whose preferred destination is not included in the outbound train’s formation requirement, a new directed edge connecting the classification track node and the outbound train node will be added to the tripartite graph. Hence, the tripartite graph is able to reflect dynamics induced by decisions on the shunting operations, i.e., the decisions (on the hump sequencing and classification track assignment) by the DRL agent, and the heuristic decisions (on the outbound train assembly and shunting locomotive scheduling), which favourably assists the DRL in learning dynamic and effective policy.

Interdependency. The tripartite graph encoded states benefit the representation of the interactions among different shunting operations in the marshalling yard. As the intermediate node set in the tripartite graph, the classification track nodes elegantly reflect the real-time railcar distribution in the classification yard, which influences the agent’s actions for the inbound train disassembly and the heuristic decisions for outbound train assembly, and vice versa. Such interdependency in dynamics is very important for learning effective policies in the DRL. For another interdependency as mentioned in Section 3.3, if a non-prioritized classification track is assigned to a block, the obstruction on this mixed-used track may happen due to the FIFO principle, which hinders the process of outbound train assembly. In the tripartite graph encoded states, the added edges during transitions can reflect this type of assignment decision (by the DRL agent) and the consequent assembly operations (by the heuristic rule), by modifying the connections between nodes.

In summary, the dynamics in the marshalling yard and the interdependency between shunting operations can be well represented in our proposed tripartite graph, via either varied features or modified edge sets. To better distinguish different states and learn the effective policy, we propose a policy network to process the graph encoded state for representation learning and action reasoning, which is elaborated below.

4.4. Policy network

The policy network consists of the representation learning and action reasoning parts, which are realized by a tripartite graph isomorphism network (TGIN) and a decision making network (DMN), respectively.

4.4.1. TGIN for representation learning

Graph neural networks (GNNs) are commonly used to automatically learn continuous vectors (i.e., embeddings) that represent nodes, edges, and graphs. By the message-passing and aggregation processes, the final embeddings derived from a GNN capture both local and global information on a graph, incorporating the interdependency between close and distant nodes/edges. These

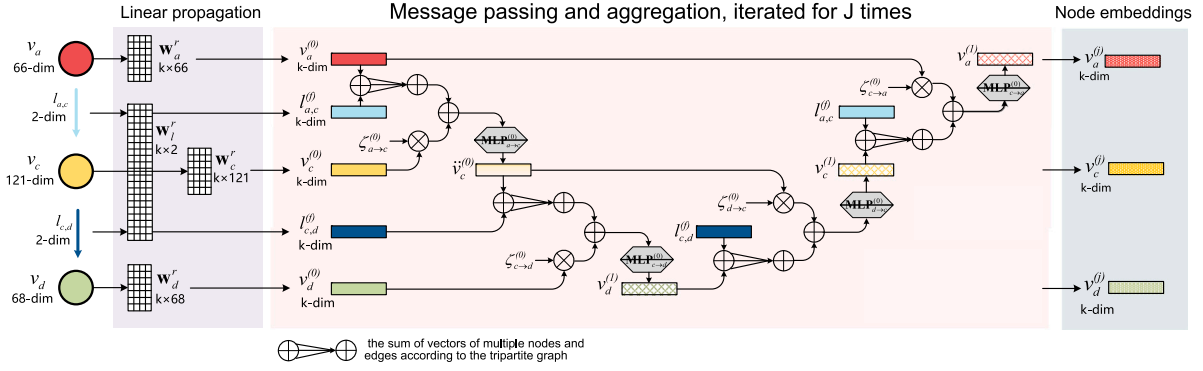


Fig. 7. The tripartite graph isomorphism network (TGIN) for node embeddings.

embeddings are beneficial for making informed decisions in complex systems (e.g., in the marshalling yard). On the other hand, GNNs can be leveraged to handle graphs of different scales, enabling applications in various scenarios (e.g., varying numbers of inbound/outbound trains in different RIO-MY problem instances).

One of the most powerful GNN types is the graph isomorphism networks (GINs), which proved to have strong discriminative capability (Xu et al., 2018). However, most GINs are only applicable to graphs without disjoint node sets. In this paper, we propose the tripartite graph isomorphism network (TGIN) to effectively learn embeddings on the tripartite graph (i.e., G_t), as shown in Fig. 7. The detailed message-passing and aggregation processes in TGIN are as follows.

Given a graph encoded state G_t , the raw feature vectors of each node and edge, i.e., v_a , v_c , v_d , $l_{a,c}$, and $l_{c,d}$, are linearly projected as below:

$$\begin{aligned} v_a^{(0)} &= \mathbf{W}_a^r v_a & v_c^{(0)} &= \mathbf{W}_c^r v_c & v_d^{(0)} &= \mathbf{W}_d^r v_d & l_{a,c}^{(f)} &= \mathbf{W}_l^r l_{a,c} & l_{c,d}^{(f)} &= \mathbf{W}_l^r l_{c,d} \\ \mathbf{W}_a^r &\in \mathbb{R}^{k \times 66}, & \mathbf{W}_c^r &\in \mathbb{R}^{k \times 121}, & \mathbf{W}_d^r &\in \mathbb{R}^{k \times 68}, & \mathbf{W}_l^r &\in \mathbb{R}^{k \times 2} \end{aligned} \quad (34)$$

where $\mathbf{W}_a^r$, $\mathbf{W}_c^r$, $\mathbf{W}_d^r$, and $\mathbf{W}_l^r$ are trainable parameters, transforming the raw feature vectors into k -dimensional initial embeddings. The unified dimension of embeddings facilitates the subsequent message passing between nodes and edges, so that it benefits the representation learning on the graph. Given the initial node and edge embeddings, the message-passing and aggregation are executed along the two edge sets in the order $\mathcal{V}_a \rightarrow \mathcal{V}_c \rightarrow \mathcal{V}_d \rightarrow \mathcal{V}_c \rightarrow \mathcal{V}_a$ (as shown in Fig. 7), where $\mathcal{V}_a \rightarrow \mathcal{V}_c \rightarrow \mathcal{V}_d$ indicates the two forward message-passing and aggregation processes, and $\mathcal{V}_d \rightarrow \mathcal{V}_c \rightarrow \mathcal{V}_a$ indicates the two backward processes. Taking the forward process from inbound train nodes $\mathcal{V}_a$ to classification track nodes $\mathcal{V}_c$ as an example, the embedding of each classification track node is computed as below,

$$\mathcal{V}_a \rightarrow \mathcal{V}_c : v_c^{(j)} = \text{MLP}_{a \rightarrow c}^{(j)} ((1 + \zeta_{a \rightarrow c}^{(j)}) \cdot v_c^{(j)} + \sum_{a \in \mathcal{N}_{in}(c)} (v_a^{(j)} + l_{a,c}^{(f)})) \quad (35)$$

where j denotes the index of the iteration, in which embeddings are updated following the order $\mathcal{V}_a \rightarrow \mathcal{V}_c \rightarrow \mathcal{V}_d \rightarrow \mathcal{V}_c \rightarrow \mathcal{V}_a$. a and c represent the inbound train node and classification track node, respectively. $\mathcal{N}_{in}(c)$ denotes the (incoming) neighbouring nodes of the classification track node c , i.e., the inbound train nodes that are connected to the classification track node c via edges in $\mathcal{L}_{a,c}^i$. In Eq. (35), we add embeddings of neighbouring nodes (i.e., $v_a^{(j)}$) with embeddings of corresponding edges (i.e., $l_{a,c}^{(f)}$), and then pass the sum to the classification track node c , as shown in Fig. 8(a). Regarding the aggregation, we integrate the sum into the node embedding of the track node c , where the trainable value $\zeta_{a \rightarrow c}^{(j)}$ is used to automatically weigh how much information is retained in the embedding of the classification track node c , i.e., $v_c^{(j)}$. Finally, a multi-layer perceptron $\text{MLP}_{a \rightarrow c}^{(j)}$ is applied to evolve the integrated embedding and attain the updated node embedding $v_c^{(j)}$ for the classification track node c . We structure $\text{MLP}_{a \rightarrow c}^{(j)}$ by two hidden layers with $2k$ dimensions, which are activated by the ReLU function, and the output layer with k dimensions.

Similarly, other message-passing and aggregation processes are formulated as below:

$$\mathcal{V}_c \rightarrow \mathcal{V}_d : v_d^{(j+1)} = \text{MLP}_{c \rightarrow d}^{(j)} ((1 + \zeta_{c \rightarrow d}^{(j)}) \cdot v_d^{(j)} + \sum_{c \in \mathcal{N}_{in}(d)} (v_c^{(j)} + l_{c,d}^{(f)})) \quad (36)$$

$$\mathcal{V}_d \rightarrow \mathcal{V}_c : v_c^{(j+1)} = \text{MLP}_{d \rightarrow c}^{(j)} ((1 + \zeta_{d \rightarrow c}^{(j)}) \cdot v_c^{(j)} + \sum_{d \in \mathcal{N}_{out}(c)} (v_d^{(j+1)} + l_{c,d}^{(f)})) \quad (37)$$

$$\mathcal{V}_c \rightarrow \mathcal{V}_a : v_a^{(j+1)} = \text{MLP}_{c \rightarrow a}^{(j)} ((1 + \zeta_{c \rightarrow a}^{(j)}) \cdot v_a^{(j)} + \sum_{c \in \mathcal{N}_{out}(a)} (v_c^{(j+1)} + l_{a,c}^{(f)})) \quad (38)$$

where $\mathcal{N}_{out}(\cdot)$ denotes the outgoing neighbouring nodes of a node. We take $\mathcal{V}_d \rightarrow \mathcal{V}_c$ as an example of the backward processes. As shown in Fig. 8(b), the embeddings of (outgoing) neighbouring outbound train nodes and corresponding edges are added, passed, and integrated on a classification node. After the message-passing and aggregation through $\mathcal{V}_a \rightarrow \mathcal{V}_c \rightarrow \mathcal{V}_d \rightarrow \mathcal{V}_c \rightarrow \mathcal{V}_a$ (i.e., Eqs. (35) to (38)), an iteration (i.e., a layer in the TGIN) is finished, and all node embeddings on the graph G_t are updated. We apply J iterations to deliver more advanced node embeddings, with the index $j = \{0, \dots, J-1\}$. In different iterations, layers of TGIN share

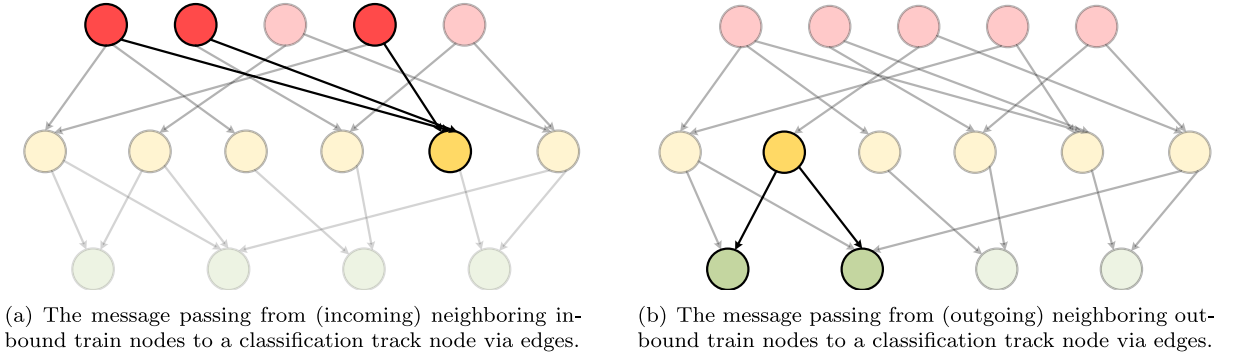


Fig. 8. The message passing from incoming and outgoing neighbour nodes and edges on the tripartite graph.

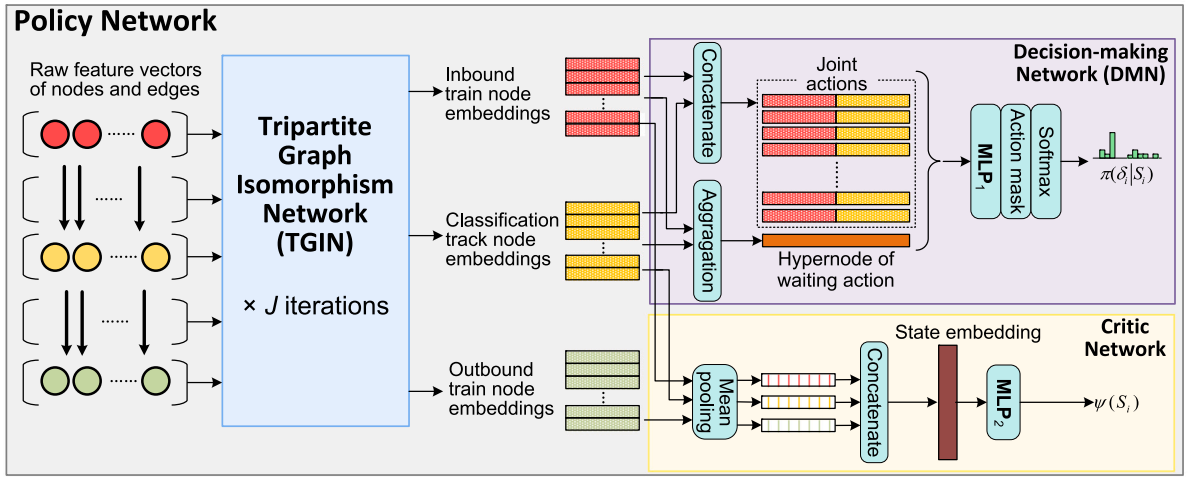


Fig. 9. The architecture of the policy network.

the same neural architecture but independent trainable parameters in Eqs. (35) to (38). We keep all embeddings derived from each iteration to be k -dimensional. After J iterations, the final node embeddings, i.e., $v_a^{(J)}, v_c^{(J)}, v_d^{(J)} \in \mathbb{R}^k$, are generated for all nodes.

It is noteworthy that we do not execute message-passing and aggregation on edges, since their information is well reflected and integrated in node embeddings which we use for the subsequent action reasoning. Meanwhile, the message-passing and aggregation processes in each layer, i.e., Eqs. (35) to (38), run in parallel for each node set. Therefore, TGIN is efficient to compute all node embeddings and applicable to RIO-MY instances with various sizes, e.g., with different numbers of inbound trains, classification tracks, and outbound trains.

4.4.2. DMN for action reasoning

Given the node embeddings derived from TGIN, we design a decision making network (DMN) to select an action $\delta_i = (a, c)$ to be executed in the environment. The overall policy network, including TGIN and DMN, is shown in Fig. 9.

Considering the available joint actions are pairs of inbound train and classification track, as mentioned in Section 4.2.3, we concatenate embeddings of an inbound train node and a classification track node to represent a joint action. More specific, we represent the joint action (a, c) by the $2k$ -dimensional embedding $[v_a^{(J)} \parallel v_c^{(J)}]$, where $\parallel$ means the concatenation operation of two vectors. $v_a^{(J)}$ and $v_c^{(J)}$ are the embeddings of inbound train node a and classification track node c , respectively, which are provided by TGIN. On the other hand, we set a hypernode that aggregates node embeddings to represent the waiting action ω . The waiting action means bypassing the immediate disassembly of any inbound train that is ready for humping, but waiting to select an inbound train that will be ready in the future states. Accordingly, we define the waiting action under the current state S_i as below,

$$\mu_i = [\frac{1}{|\rho_a^i|} \sum_{a \in \rho_a^i} v_a^{(J)} \parallel \frac{1}{|\mathcal{V}_c|} \sum_{c \in \mathcal{V}_c} v_c^{(J)}] \quad \rho_a^i \subset \mathcal{V}_a \quad (39)$$

where μ_i is the hypernode embedding. ρ_a^i denotes a subset of $\mathcal{V}_a$, containing all inbound trains that are not ready for humping under the state S_i , i.e., the trains that have not arrived and the trains that have arrived but the arrival technical inspection is not finished. In Eq. (39), the embeddings of nodes in ρ_a^i and $\mathcal{V}_c$ are averaged, respectively, and then are concatenated to attain the embedding μ_i .

In doing so, the waiting action is also represented by a $2k$ -dimensional embedding. All embeddings of (joint and waiting) actions are fed into a multi-layer perceptron, as below,

$$\begin{aligned} h((a, c)) &= \mathbf{MLP}_1([v_a^{(J)} \parallel v_c^{(J)}]) \quad \forall a \in \mathcal{V}_a, c \in \mathcal{V}_c \\ h(\omega) &= \mathbf{MLP}_1(\mu_i) \end{aligned} \quad (40)$$

where $\mathbf{MLP}_1$ is a multi-layer perceptron structured by two $4k$ -dimensional hidden layers, which are activated by the Tanh function, and a single-value output. The output value represents the score of selecting each action.

According to the available action space Δ_{S_i} under the state, as defined in Section 4.2.3, we remove the scores of infeasible actions, and then compute probabilities of selecting each available action by the Softmax function, such that,

$$P(\delta_i) = \frac{\exp(h(\delta_i))}{\sum_{\delta' \in \Delta_{S_i}} \exp(h(\delta'))} \quad \delta_i \in \Delta_{S_i} \quad (41)$$

where $P(\delta_i)$, $\delta_i \in \Delta_{S_i}$ denotes the probability distribution over all available actions under state S_i , from which an action is sampled. Until now, the policy network π_θ , including TGIN and DMN, is established by $\pi_\theta(\delta_i|S_i) = P(\delta_i)$, where θ denotes all trainable parameters in the policy network.

4.5. Policy Optimization

We apply the Proximal Policy Optimization (PPO) algorithm (Schulman et al., 2017) with Adam optimizer to train the policy network π_θ . PPO is a commonly used actor-critic DRL algorithm, which has been applied to broad applications, including robotics, game playing, combinatorial optimization, etc. In our RIO-MY problem, the actor is the policy network π_θ , and the critic estimates the cumulative reward associated with taking actions under π_θ . In the following content, we elaborate the neural network to parameterize the critic, and the training procedure by PPO algorithm.

Critic network. Despite building an independent critic network, we process the node embeddings from TGIN by an additional sub-network to estimate the value function, i.e., cumulative rewards under states. As shown in Fig. 9, given a state S_i , we first attain the state embedding by averaging the node embeddings (from TGIN) in each node set and concatenating the averaged embeddings. Then a multi-layer perceptron is used to process the state embedding for estimating the cumulative reward under S_i . Formally, the sub-network is expressed as below,

$$\psi_\phi(S_i) = \mathbf{MLP}_2[\frac{1}{|\mathcal{V}_a|} \sum_{a \in \mathcal{V}_a} v_a^{(J)} \parallel \frac{1}{|\mathcal{V}_c|} \sum_{c \in \mathcal{V}_c} v_c^{(J)} \parallel \frac{1}{|\mathcal{V}_d|} \sum_{d \in \mathcal{V}_d} v_d^{(J)}] \quad (42)$$

where ϕ denotes trainable parameters in the sub-network. The mean pooling is applied to the embeddings of inbound train nodes, classification track nodes, and outbound train nodes, respectively. The resulting three k -dimensional vectors are concatenated to constitute the state embedding of S_i . The multi-layer perceptron $\mathbf{MLP}_2$ is similar to the structure of $\mathbf{MLP}_1$, except that $\mathbf{MLP}_2$ has a different input dimension (i.e., $3k$).

Training procedure. The training procedure by PPO algorithm is presented in Algorithm 1. As mentioned above, the actor is the policy network π_θ to select an action, and the critic is the sub-network $\psi_\phi(S_i)$ to estimate the value function. Please note that, in each training step, R instances of the RIO-MY problem are processed by the neural networks in parallel, which can be easily realized by GPU with high efficiency. Regarding each instance, the action δ_i sampled by π_θ is executed under the state S_i in the environment, and then the state transits to S_{i+1} . The above agent-environment interaction continues until the terminal state. All the collected trajectories in a training episode are then divided into multiple mini-batches, which are repeatedly used to train the neural networks for K times. We use the general losses in PPO and update the neural networks by the Adam optimizer. To improve training stability and achieve better policy, we decay the learning rate every d_f steps. We also validate the policy network on a set of instances every v_f steps.

Algorithm 1 Training procedure by PPO

Require: Policy network with trainable parameters θ ; critic network with trainable parameters ϕ ; reward discount factor γ ; maximum training step O ; the number of instances R used in each training step; the number of epochs K ;

```

1: for  $o = 1, 2, \dots, O$  do
2:   Generate  $X$  new instances.
3:   for  $r = 1, 2, \dots, R$  do ▷ Executed in parallel
4:     Initialize the start state  $S_0$  based on instance  $r$ .
5:     Initialize the tripartite graph  $\mathcal{G}_i$ .
6:     while  $S_i$  is the not terminal state do
7:       Extract node embeddings based on  $\mathcal{G}_i$ .
8:       Select action  $\delta_i$  by sampling using  $\pi(\cdot)$ .
9:       Move to next state  $S_{i+1}$ .
10:      Calculate reward  $R(S_i, \delta_i, S_{i+1})$ .
11:       $S_i = S_{i+1}$ .
12:      Update  $\mathcal{G}_i$ .
13:     end while
14:   end for
15:   Calculate returns-to-go (i.e., the discounted future rewards) for episodes of each instance.
```

```

16:   Using Generalized Advantage Estimation (GAE) to calculate the estimated advantages based on the current  $\psi_\phi$ .
17:   Normalize the advantages  $\bar{A}$ .
18:   for  $k = 1, 2, \dots, K$  do
19:       Calculate PPO loss using the gradient-clip strategy. ▷ repeated based on mini-batch
20:       Optimize the trainable parameters  $\theta$  and  $\phi$  using Adam optimizer.
21:   end for
22:   Update the network parameters.
23:   if  $o \bmod d_f = 0$  then
24:       Decay the learning rate.
25:   end if
26:   if  $o \bmod v_f = 0$  then
27:       Validate the policy network.
28:   end if
29: end for

```

Last but not least, during validation and testing, we sample actions using π_θ under each state, same as in training. This sampling strategy can provide more possibilities to find better solutions than only selecting the action with the highest probability. To this end, we solve multiple copies of each instance in parallel during validating or testing. Then, the best one from the solutions (to the multiple copies) is picked as the final solution to the instance. The parallel solving of (a batch of) instances for the training in the above algorithm is also applied to solve multiple copies of each instance in validation and testing, which significantly improves the computational efficiency.

5. Experiments

In this section, the proposed DRL method for the RIO-MY problem is tested on real-world instances. The experiment settings are introduced in Section 5.1. Then, the ablation study is presented in Section 5.2, which focuses on reward shaping, the hypernode of waiting action, the comparison of different neural networks in node embedding, and stability evaluation by setting random seeds. From Sections 5.3 to 5.5, the performance, generalization, and extension of the proposed method are clarified, respectively.

5.1. Experiment settings

5.1.1. Instance generation

This paper presents a case study of two real-world railway marshalling yards in China, namely M1 and M2. The number of infrastructures and equipment in them are fixed and shown in Table 3. M1 is a relatively small marshalling yard with 10 classification tracks, while M2 is a larger yard with 20 classification tracks. As we stated before, the number of possible railcar destinations within the marshalling yards is fixed. For each classification track, the preferred destination is given, as shown in Table 4. Note that certain classification tracks may share the same preferred destinations, such as $c3$ and $c4$ in marshalling yard M2.

Since a substantial number of instances are required for training, validating, and testing the DRL method, we have collected four real-world instances to serve as problem scenarios to generate diverse sets of synthetic instances. Specifically, as shown in Table 5, M1-R1 and M1-R2 are two real-world instances of the marshalling yard M1. M1-R1 has 11 inbound trains, 10 outbound trains, and 562 railcars, while M1-R2 consists of 10 inbound trains, 10 outbound trains, and 546 railcars. They are collected from two independent stage plans, respectively. A stage plan covers 4 h, which is the minimum planning period in railway marshalling yards in China. In this planning period, using 00:00 as the start time, inbound trains arriving from 00:00 to 04:00 are disassembled, and the outbound trains that depart from 02:00 to 06:00 are targets of assembly. Hence, the end time of the planning period used to calculate the railcar dwell time is 06:00. These two instances are classified as regular-size instances. M2-L and M2-H are two real-world instances from the marshalling yard M2. M2-L is a large-size instance, which covers 8 h with 17 inbound trains, 16 outbound trains, and 887 railcars, while M2-H is a huge-size instance, which covers 12 h with 26 inbound trains, 24 outbound trains, and 1324 railcars. When generating synthetic instances, the scheduled arrival and departure times of trains and the number of railcars in each individual block are sampled randomly from uniform distributions. The parameters of these uniform distributions are determined by the corresponding original values. For example, when generating instances based on M1-R1, the arrival time of an inbound train (u_a^{in}) is sampled from the uniform distribution $U(u_a^{in} - 60, u_a^{in} + 60)$. For a block β , the number of railcars is sampled $U(0.8k_\beta, 1.2k_\beta)$, where k_β is the number of railcars in block β . The stochastic ranges of each uniform distribution are shown in Table 5. During the training processes, the instances are generated on-the-fly based on the above distributions of a single problem scenario. Before training, 50 validation instances and 50 testing instances are generated based on the same scenario, respectively.

5.1.2. Experiment schemes, benchmarks, and evaluation indicators

Experiment schemes. The experiments are set as the following four parts. **(1) Ablation Study:** An ablation study is presented to validate the effectiveness of the MDP modelling and the TGIN based policy network. **(2) Solving Performance:** To investigate the solution quality, two DRL agents (i.e., policy networks) are trained based on two disparate scenarios M1-R1 and M2-L, respectively. Each trained policy network is used to solve the 50 testing instances and the real-world instance of its training scenario. **(3) Generalization Study:** To evaluate the two trained policy networks on scenarios that they have never been trained with, they are leveraged to directly solve instances of scenario M1-R2 and M2-H without retraining. **(4) Disturbance Handling:** We demonstrate that our method is capable of handling randomness and disruptions in the situation where the disassembly process of an inbound train is disturbed.

Table 3

The number of infrastructures and equipment in the railway marshalling yards.

Yard name	Berthing tracks (receiving yard)	Classification tracks	Berthing track (departure yard)	Destinations	Hump	Hump shunting locomotives	Pull-out shunting locomotives
M1	11	10	10	10	1	2	2
M2	14	20	12	15	1	2	2

Table 4

The preferred destination of classification tracks.

(a) Railway marshalling yard M1. $e1 \sim e10$ indicate 10 destinations.										
Classification track	$c1$	$c2$	$c3$	$c4$	$c5$	$c6$	$c7$	$c8$	$c9$	$c10$
Preferred destination	$e1$	$e2$	$e3$	$e4$	$e5$	$e6$	$e7$	$e8$	$e9$	$e10$

(b) Railway marshalling yard M2. $e1 \sim e15$ indicate 15 destinations.																				
Classification track	$c1$	$c2$	$c3$	$c4$	$c5$	$c6$	$c7$	$c8$	$c9$	$c10$	$c11$	$c12$	$c13$	$c14$	$c15$	$c16$	$c17$	$c18$	$c19$	$c20$
Preferred destination	$e1$	$e2$	$e3$	$e3$	$e4$	$e5$	$e6$	$e7$	$e7$	$e8$	$e8$	$e9$	$e10$	$e10$	$e11$	$e12$	$e13$	$e13$	$e14$	$e15$

Table 5

Real-world instances and distributions for generating synthetic instances.

Real-world instance	Planning period (hours)	Size	Number of inbound and outbound trains	Distribution for scheduled time ($\pm min$)	Distribution for railcar number
M1-R1	4	Regular	11 inbound, 10 outbound	$U(\tau_a^{in/out} - 60, \tau_a^{in/out} + 60)$	$U(0.8k_\beta, 1.2k_\beta)$
M1-R2	4	Regular	10 inbound, 10 outbound	$U(\tau_a^{in/out} - 60, \tau_a^{in/out} + 60)$	$U(0.8k_\beta, 1.2k_\beta)$
M2-L	8	Large	17 inbound, 17 outbound	$U(\tau_a^{in/out} - 90, \tau_a^{in/out} + 90)$	$U(0.7k_\beta, 1.3k_\beta)$
M2-H	12	Huge	27 inbound, 26 outbound	$U(\tau_a^{in/out} - 90, \tau_a^{in/out} + 90)$	$U(0.7k_\beta, 1.3k_\beta)$

Benchmarks. To show the effectiveness, efficiency, and quality of our DRL method, we compare it with the following methods: first-come-first-service (FCFS) heuristic rule, metaheuristics (including particle swarm optimization (PSO) and genetic algorithm (GA)), and the MINLP model presented in Section 3.2.

The FCFS rule is the most widely adopted rule in practice. Specifically, the humping sequence of inbound trains is identical to their arrival order. For each block, the first available classification track is assigned. The heuristic rules of outbound train assembly and shunting locomotive scheduling are also applied here.

Since there are no existing PSO and GA algorithms that can be used to solve the RIO-MY problem directly, we design a two-stage simulation-optimization framework. In the framework, the discrete-event simulation model is used to compute the fitness function of PSO and GA, while PSO and GA only solve the two crucial decisions, identical to the role of the policy network in the proposed DRL method. In doing so, we can make a fair comparison between methods. The parameters of GA and PSO, such as the iterations and the population size, are kept the same among all experiments. More details can be found in Appendix D.

The MINLP model is solved by the commercial solver Gurobi9.0. As it is computationally very expensive to solve the whole MINLP model directly, we decompose the MINLP model into three sub-models. The first one is to obtain an ideal solution for the humping and assembly sequences and the train makeup plan. The second sub-model adjusts the assignment and groups railcars into blocks based on the solved humping and assembly sequences. Finally, the third sub-model assigns classification tracks for all blocks, considering the principle of first-in-first-out and using prioritized classification tracks first. We use Gurobi9.0 to solve these three sub-models sequentially to obtain solutions for instances. We name this method *decom-MINLP*. More details can be found in Appendix E.

Evaluation indicators. For a single instance, the result of the average dwell time of all railcars obtained by the *decom-MINLP* is treated as the baseline solution. Hence, for each method, we compute the performance gap of its solution to the solution obtained by the *decom-MINLP* as below:

$$Gap(x) = (x - Res_{MINLP}) / Res_{MINLP} \times 100\% \quad (43)$$

where Res_{MINLP} and x are the average railcar dwell time obtained by the *decom-MINLP* model and another solving method, respectively. Therefore, for each problem scenario, we can evaluate a solving method (i.e., our DRL method or other benchmarks) based on its synthetic testing instances in terms of the following performance indicators:

- F_{test} , the average value of the average railcar dwell time of all 50 testing instances.
- Gap_{test} , the average gap of the 50 testing instances, between the average railcar dwell time and the baseline solutions obtained by *decom-MINLP*.
- Q_{ave} , the average time of solving each testing instance.

Furthermore, to highlight the performance in practical situations, the single real-world instance is also tested individually and evaluated by:

- F_{real} , the average railcar dwell time of the real-world instance.
- Gap_{real} , the gap of the real-world instance, between the average railcar dwell time and the baseline solution obtained by decom-MINLP.

It should be emphasized that, as we stated in Section 4.5, in our DRL method, the sampling strategy for action selection is applied in validation and testing. When validating or testing a single instance using a trained agent, 50 copies of the instance are solved in parallel. The best one of the 50 solutions, i.e., the one with the minimum average railcar dwell time, is selected as the solution to the instance. Therefore, the solving time of a single instance using the DRL method means the time of solving all 50 copies in parallel. In addition, although the changing train operation parameters in the DRL method are fixed in the decom-MINLP model, their fixed values are determined according to real-world values. Hence, the gap can still reflect the quality of solutions obtained by the DRL method approximately. Furthermore, some detailed operations, such as crew shift changes and railcar resorting, are necessary for real-world management but are not included in the simulation model of this work. Hence, the direct comparison between our solutions and the historical data is excluded to avoid unfair comparative results.

5.1.3. Train operational parameters

Several train operational parameters are predetermined according to reality as follows. The technical inspection times for arrival and departure i.e., t_a^{tech} and t_d^{tech} , are 30 min and 20 min, respectively. The safety margin of the hump t_h^{sm} is 5 min. The minimum and maximum number of railcars assembling an outbound train, λ_d^{min} and λ_d^{max} , are 40 and 60. All outbound trains comply with these two limits. The capacity of all classification tracks κ_c is 60 railcars. Some train operational parameters are changing with real-time situations and vary in different marshalling yards. For example, the required movement time of a block roll into the classification yard is related to the number of existing railcars on the assigned classification track and the layout of classification tracks. For these parameters, we directly calculate them in real-time in the discrete-event simulation model.

5.1.4. Hyperparameter configuration

With preliminary experiments, we configure the hyperparameters as follows. The number of layer iterations in the TGIN is set as $J = 3$. The unified dimension throughout the policy network is set as $k = 128$. The reward discount factor is set as $\gamma = 1$. For the PPO algorithm, the maximum training step is set to 1000. The number of parallel instances is set as $R = 10$, which means 10 instances are learned simultaneously in each step. The epochs of repeatedly utilizing samples in each step is set as $K = 3$, which is executed based on 64-size mini-batches. The ratio of the gradient clip in the loss function is set to 0.2. The coefficients of the policy loss, value loss, and entropy term are set to 1, 0.5, and 0.01, respectively. The trainable parameters are optimized using Adam. The learning rate is set to $lr = 0.0001$, and is decayed by the factor 0.95 every 50 training steps. The model is trained and tested on a PC with a single Nvidia GeForce 1080Ti GPU.

5.2. Ablation study

The ablation study aims to evaluate different designs in the proposed architecture to perfect our DRL method for the RIO-MY problem.

5.2.1. The estimation in reward function and the aggregation pattern of hypernode

In this part, we focus on two design choices in the proposed framework: the computation of estimated railcar dwell time in the reward function and the aggregation pattern of the hypernode for the waiting action in the policy network. As the reward function stated in Section 4.2.5, an outbound train needs to be selected as the estimated assignment for *pending railcars* to compute the estimated railcar dwell time. Here we provide two principles: choosing the available outbound train with the earliest or the one with the latest scheduled departure time, named *ES* and *LS*, respectively. What drives us to design *ES* is that the heuristic rule of assembling outbound trains in Algorithm A.1 attempts to assemble outbound trains following the order of scheduled departure time. On the other hand, choosing the latest available outbound train is possible to provide more positive value rewards during transitions in an episode. The aggregation for hypernode is realized as Eq. (39) in Section 4.4.2. Based on it, another multi-layer perceptron (MLP) is added after the concatenation, i.e., $\mu_i = \text{MLP}_3[\frac{1}{|\rho_a|} \sum_{a \in \rho_a^i} v_a^{(J)} \parallel \frac{1}{|\mathcal{V}_c|} \sum_{c \in \mathcal{V}_c} v_c^{(J)}]$. MLP_3 has two 4k-dimensional hidden layers and a Tanh activation. We name this new structure as *NM* and the original structure as *Ori*.

We enrich the ablation study by four combinations of the above designs. The experiments are conducted on the testing instances and the real-world instance of scenario M1-R1. The results are shown in Table 6. We can see that the design *ES+Ori* and *LS+NM* can both obtain the shortest average railcar dwell time on the real-world instance, but *ES+Ori* can obtain a smaller average value on testing instances, i.e., the F_{est} . Hence, we conclude that it outperforms other designs, implying its ability to obtain solutions with higher qualities on general instances can be expected. Therefore, our DRL method is finalized by choosing the earliest available outbound train in the reward function and generating the hypernode embedding using Eq. (39). This architecture is adopted in all the following experiments.

Table 6

Performance for different designs of reward function and hypernode aggregation, based on scenario M1-R1.

	$ES + NM$	$ES + Ori$	$LS + NM$	$LS + Ori$
F_{test}	13750 s	13655 s	13711 s	13774 s
F_{real}	13335 s	13280 s	13280 s	13451 s

Table 7

Performance of different neural networks for node embedding, based on scenario M1-R1.

	GAT	GCN	MLP	TGIN (our method)
F_{test}	13948 s	13848 s	14014 s	13655 s
F_{real}	13425 s	13425 s	13667 s	13280 s

5.2.2. Neural networks for node embedding

Based on the finalized reward function and aggregation pattern of hypernode, in this section, we compare several neural components for node embedding in the policy network. Specifically, we use Graph Convolution Network (GCN), Graph Attention Network (GAT), and Multi-Layer Perceptron (MLP), which are widely-used and powerful neural networks, to replace the TGIN in the policy network. The experiments are also conducted on the testing instances of scenario M1-R1. Similar to Table 6, the average value of railcar dwell time of all 50 instances, i.e., F_{test} , and the railcar dwell time of the real-world instance, i.e., F_{real} , are used to evaluate the neural networks. The results are shown in Table 7. It is obvious that the proposed TGIN architecture, which is customized for the RIO-MY problem, outperforms other standard neural networks, according to the average performance of testing instances and the performance of the single real-world instance. Therefore, the TGIN plays a significant role in obtaining informative node embeddings and contributes advantages to the entire DRL method.

5.2.3. Random seeds

In this section, to evaluate the stability and reliability of the proposed DRL method, we use random seeds to conduct multiple times of training. The instances used to train the agent will be changed completely when every different seed is used. The random exploration during the training processes also differs under different seeds. The validation instances are still generated using the fixed seed before training, which enables reproducible validation instances, allowing for a fair comparison of the performance under different seeds.

We set five random seeds in the experiments. The training and validation processes on scenario M1-R1 are shown in Fig. 10. Under each seed, the training curve is plotted by the average value of the average railcar dwell time of paralleled 10 instances in each training step. The validation curve is plotted every 50 training steps. Each validation result is the average value of the average railcar dwell time of the 50 validation instances. It can be observed that, under 5 different seeds, the training processes are similar, towards convergence after about 400 steps. The best validation results under different seeds are also almost the same. Therefore, the proposed DRL method has stable performance under different random seeds, and we choose to use a fixed seed (seed 1) in all the following experiments.

5.3. Performance on M1-R1 and M2-L

In this section, we demonstrate the performance and efficiency of the proposed DRL method in solving the RIO-MY problem. More specifically, we train two agents using the synthetic instances generated from scenarios M1-R1 and M2-L, respectively. The trained agents are then evaluated on the synthetic testing instances and real-world instances of M1-R1 and M2-L.

5.3.1. Performance on synthetic testing instances

We first train a DRL agent on scenario M1-R1. After the training is completed, the trainable parameters that obtained the best results on the validation set are saved for the agent. Then, we use this trained agent to solve the testing instance of M1-R1. The results are shown in the upper half of Table 8. For the 50 testing instances, the average value of the average railcar dwell time (F_{test}) is 13655 s. It outperforms the FCFS rule and the heuristic algorithms. The average gap between solutions obtained by our DRL method and the solutions from the decom-MINLP model (Gap_{test}) is only 3.93%, demonstrating the DRL method can obtain high-quality solutions. As for the solving time, the DRL method only needs an average less than 10 s to solve an instance, which is much shorter than heuristic algorithms and the decom-MINLP model. It should be emphasized that the listed solving time of the MINLP model is calculated based on the decomposition method, which is the reason for being shorter than solving using heuristic algorithms. The solving time of using the FCFS rule is extremely short, since the simulation is a discrete-event model without any entity modelling. Such a short simulation time means a very low time cost of action execution, reward computation, and state transitions. This enables us to integrate the simulation model in the DRL method with little impairment of training efficiency.

To test the DRL method in solving larger instances in a different marshalling yard, we train another agent with the instances generated from M2-L. M2-L is a large-size scenario than M1-R1, with more trains, railcars, destinations, tracks, and longer planning periods. The training and validation processes are similar to Fig. 10. The training time is 6.5 h, the trained agent with the best

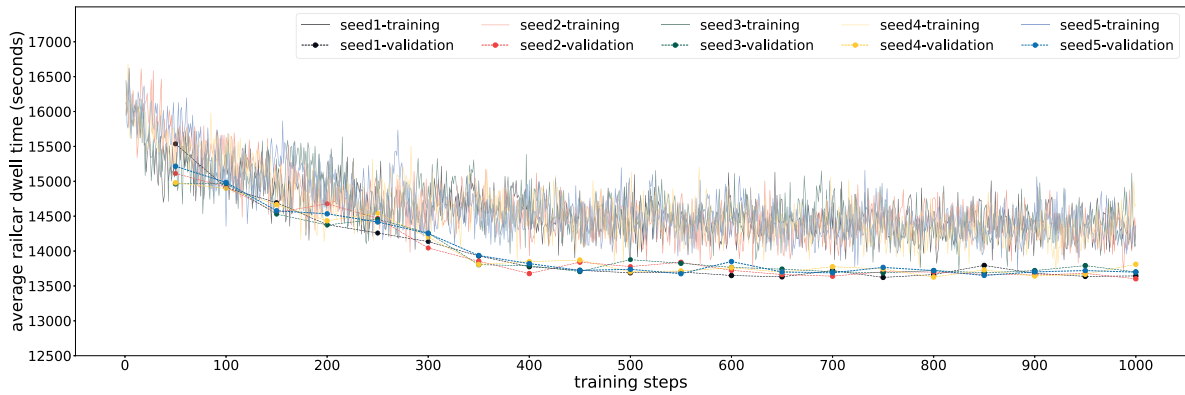


Fig. 10. The training and validation processes of the scenario M1-R1 with five random seeds.

Table 8

Performance on testing instances of scenario M1-R1 and M2-L.

Scenario	Indicator	DRL	FCFS	GA	PSO	decom-MINLP
M1-R1	F_{test}	13 655 s	15 169 s	14 114 s	13 961 s	13 139 s
	Gap_{test}	3.93%	15.46%	7.45%	6.29%	–
	Q_{ave}	9.14 s	0.03 s	123.29 s	118.71 s	46.44 s
M2-L	F_{test}	17 682 s	18 859 s	18 609 s	18 258 s	16 449 s
	Gap_{test}	7.52%	14.71%	13.28%	11.17%	–
	Q_{ave}	22.25 s	0.08 s	294.79 s	310.25 s	89.10 s

performance on the validation set is used to solve the testing instances of M2-L. The results of all methods are shown in the lower half of Table 8. For the 50 testing instances, the average value of the average railcar dwell time (F_{test}) of the DRL method is 17 683 s, with an average gap (Gap_{test}) of 7.52% to the solutions from the decom-MINLP model. Similar to the results on M1-R1, our DRL method obtains better solutions than the FCFS rule and heuristic algorithms, especially considering its shorter solving time than the heuristic algorithms. Compared to the case of M1-R1, the solving time of our DRL method increases as more node embeddings need to be computed by the policy network, and episodes with more transitions need to be solved. Nevertheless, the DRL method takes only about a quarter of the solving time of the decom-MINLP model.

5.3.2. Performance on the real-world instance and solution analysis

To show the performance in practical situations and illustrate detailed railcar itineraries, the real-world instances of scenario M1-R1 and M2-L are solved by the trained agents. The results of railcar dwell time are shown in Table 9. Similar to the conclusions we draw from the synthetic testing instances, for the real-world instance of the regular-size scenario M1-R1, the average railcar dwell time obtained by the DRL method (F_{real}) is 13 280 s, with a gap of 3.02% to the solution from the decom-MINLP model (Gap_{real}), outperforming FCFS, GA, and PSO.

For the real-world instance of the large-size scenario M2-L, the results show that the average railcar dwell time (F_{real}) is 17 295 s. The corresponding gap to the solution from the decom-MINLP model (Gap_{real}) is 6.45%. Such a gap is larger than that under the real-world instance of scenario M1-R1. Whereas, it still outperforms the FCFS rule and the two heuristic algorithms. Furthermore, we notice that the simple FCFS rule already performs well, with a 10.95% gap and an extremely short solving time. Compared to the FCFS rule, the solutions from GA and PSO only decrease the gap by about 1.5%. On the contrary, the proposed DRL method can decrease the gap by 4.5%, significantly improving the quality of the solution.

Analysis of generated solution. To gain insights into the policy learned by the DRL agent, we analyse the details of the generated railcar itinerary of the real-world instance of M1-R1, which is the solution with the 13 280 s average railcar dwell time in Table 9. For inbound trains, the scheduled arrival time, block composition, and the solution of humping sequences and humping time are shown in Table 10. The inbound trains arrive frequently in the first two hours. It leads to a recurrent situation in which more than one inbound train is ready to be humped under a state, so a reasonable decision of humping sequence is highly required. The first inbound train, $a1$, is humped at 00:38, right after the arrival technical inspection. The total time of pushing it to the hump and disassembling all blocks from it is 17 min. Then, after the 5-min hump safety margin, the next humping and disassembly process begins at 01:00. The next inbound train to be humped is $a3$. This is an outcome of hump sequencing, which also means the humping for inbound train $a2$ has been decided to be postponed. We note that the hump is always being occupied from 00:38 to 04:46, except for the safety margins between two adjacent occupations. All inbound trains are humped and disassembled continuously without any redundant time. It means handling 11 inbound trains for a 4-h stage plan is a very heavy task. Compared to the original arrival sequence, the finalized humping sequence adjusts the inbound trains $a2$, $a6$, and $a9$.

Table 9

Performance on the real-world instances of scenario M1-R1 and M2-L.

Scenario	Indicator	DRL	FCFS	GA	PSO	decom-MINLP
M1-R1	F_{real}	13 280 s	14 109 s	13 464 s	13 610 s	12 890 s
	Gap_{real}	3.02%	9.46%	4.45%	5.59%	–
M2-L	F_{real}	17 295 s	18 026 s	17 746 s	17 810 s	16 246 s
	Gap_{real}	6.45%	10.95%	9.23%	9.62%	–

Table 10

The information of inbound trains and the solution for humping sequence (M1-R1).

Inbound train	Scheduled arrival time ^a	Railcars composition	Humping sequence index	Humping start time	Disassembly end time
a_1	00:08	[$e_3/15$], [$e_5/17$], [$e_7/6$] ^b	1	00:38	00:55
a_2	00:15	[$e_5/4$], [$e_2/9$], [$e_7/20$]	11	04:31	04:46
a_3	00:22	[$e_7/16$], [$e_{10}/16$], [$e_3/24$]	2	01:00	01:18
a_4	00:29	[$e_7/30$], [$e_4/4$], [$e_5/11$]	3	01:23	01:41
a_5	00:35	[$e_7/7$], [$e_8/12$], [$e_{10}/6$], [$e_2/22$], [$e_4/7$]	5	02:08	02:28
a_6	00:50	[$e_9/2$], [$e_8/9$], [$e_2/14$], [$e_4/29$]	4	01:46	02:03
a_7	01:01	[$e_9/30$], [$e_5/10$], [$e_2/17$]	6	02:33	02:52
a_8	01:20	[$e_4/13$], [$e_5/20$], [$e_{10}/17$], [$e_6/5$]	7	02:57	03:17
a_9	01:35	[$e_6/32$], [$e_3/17$], [$e_{10}/6$]	10	04:06	04:26
a_{10}	01:42	[$e_3/20$], [$e_6/6$], [$e_7/5$]	8	03:22	03:39
a_{11}	01:58	[$e_6/10$], [$e_3/16$], [$e_7/25$]	9	03:44	04:01

^a We designate 00:00 as the start time of the planning period and thereafter convert the real-world time values accordingly.^b [e/n] is a block containing n railcars and destination is e .

For outbound trains, the scheduled departure time and the formation requirement of destinations are shown in Table 11. The solutions of the assembly sequence and train makeup plan are also presented. For example, the finalized formation of the outbound train d_3 is three blocks containing 23 railcars, 16 railcars, and 17 railcars, respectively. They come from the inbound train a_1 , a_3 , and a_4 , respectively. Except for blocks from inbound trains, two blocks that already existed in the classification yard at the beginning of the planning period are also assigned to outbound trains d_2 and d_7 , respectively. In total, 10 of 11 outbound trains have been assembled and then depart on time. The outbound train d_1 is cancelled, since there are no sufficient railcars with destination e_1 . The heuristic rule of scheduling pull-out shunting locomotives integrated in Algorithm A.1 also plays a role in determining railcar itineraries. For example, block [$a_9/e_3/17$] is a component of outbound train d_{11} . After this block stops on the classification track at 04:27, the formation of d_{11} is determined, and the assembly is supposed to start. However, at this moment, the assembly of outbound trains d_9 and d_{10} is still in process. Since only two pull-out shunting locomotives can be used, the assembly of outbound train d_{11} has to start at 04:42, instantly after a locomotive completes the assembly of outbound train d_9 .

The classification track assignment for each block during the planning period is visualized in Fig. 11. In each sub-figure of a single classification track, the upper half is the Gantt chart of all blocks using the classification track. A red/green bar in it indicates the occupation record of a block using this track as a (non-)prioritized classification track. An occupation period bar covers the time from a block begins to roll into the classification track to the block is completely pulled out. The lower half is the step chart illustrating the real-time number of railcars on the classification track. Based on Tables 10, 11, and Fig. 11, the specific spatial and temporal trajectory of a block and railcars composing it in the marshalling yard can be summarized easily, including the decomposition of a block. For example, [$e_7/30$] is a block from inbound train a_4 , which begins to be humped at 01:23. As the first block of a_4 , it is disassembled and begins to roll into its prioritized classification track c_7 at 01:36. However, at that time, there are two blocks containing 39 railcars in total already parking on c_7 . The remaining available capacity is 21, as shown in Fig. 11. Therefore, block [$e_7/30$] is divided into 2 blocks. The first is [$a_4/d_3/e_7/21$] assigned to the prioritized classification track c_7 . The other is [$a_4/d_9/e_7/9$]. The DRL agent decides to assign the non-prioritized classification track c_3 to this block. Therefore, in the sub-figure of classification track c_3 , a red bar indicates a block with destination e_7 occupies the track from 01:39 to 04:21. Such an assignment has no negative impact on succeeding blocks rolling into classification track c_3 . The classification track c_3 , as the prioritized classification track for blocks with destination e_3 , always provides sufficient capacity for succeeding blocks before 04:21.

In summary, under the real-world instance of scenario M1-R1, the DRL method can obtain high-quality railcar itinerary solutions with short railcar dwell time by reasonable decisions for the train shunting operations and the train makeup plan.

Analysis of the performance gaps. We show in Tables 8 and 9 that the solutions of the proposed DRL method have small gaps to the ones from the decom-MINLP model, and the gaps increase under scenario M2-L compared to M1-R1. In this section, we use the real-world instance of M2-L as an example to get insights into the reason for the performance gap of our DRL method.

First of all, the performance gap of the DRL method lies in the fact that the heuristic rule of assembling outbound trains in the DRL method does not wait for more blocks. More specifically, after a block rolls into the classification yard, according to Algorithm A.1, if available blocks for an outbound train under the current state reach the minimum railcar number limit, the formation of the

Table 11

The information of outbound trains and the solution for car-to-train assignment and assembly sequence (M1-R1).

Outbound train	Scheduled departure time	Destination requirement	Actual formation	Assembly sequence index	Assembly start time	Assembly end time
$d1$	02:45	$e1$	Cancelled	–	–	–
$d2$	03:08	$e3$	$[-/e3/5]$, $[a1/e3/15]$, $[a3/e3/24]^a$	1	01:22	01:49
$d3$	03:27	$e7$	$[a1/e7/23]$, $[a3/e7/16]$, $[a4/e7/21]$	2	01:39	02:08
$d4$	03:39	$e5, e10$	$[a1/e5/17]$, $[a4/e5/11]$, $[a3/e10/16]$	3	01:49	02:26
$d5$	03:53	$e2$	$[a6/e2/14]$, $[a5/e2/22]$, $[a7/e2/17]$	6	02:58	03:27
$d6$	04:10	$e4$	$[a4/e4/4]$, $[a6/e4/29]$, $[a5/e4/7]$	4	02:31	02:58
$d7$	04:25	$e8, e9$	$[a6/e8/9]$, $[a5/e8/12]$, $[-/e9/2]$, $[a6/e9/2]$, $[a7/e9/30]$	5	02:51	03:25
$d8$	04:46	$e5, e10$	$[a7/e5/10]$, $[a8/e5/20]$, $[a5/e10/6]$, $[a8/e10/17]$	7	03:25	03:59
$d9$	05:17	$e7$	$[a4/e7/9]$, $[a5/e7/7]$, $[a10/e7/5]$, $[a11/e7/25]$	8	04:06	04:42
$d10$	05:20	$e6$	$[a8/e6/5]$, $[a10/e6/6]$, $[a11/e6/10]$, $[a9/e6/32]$	9	04:23	04:50
$d11$	05:55	$e3$	$[a10/e3/20]$, $[a11/e3/16]$, $[a9/e3/17]$	10	04:42	05:08

^a $[a/e/n]$ is a block containing n railcars from inbound train a and destination is e . ‘–’ means this block is not from an inbound train, it already exists in the classification yard at the beginning of the planning period.

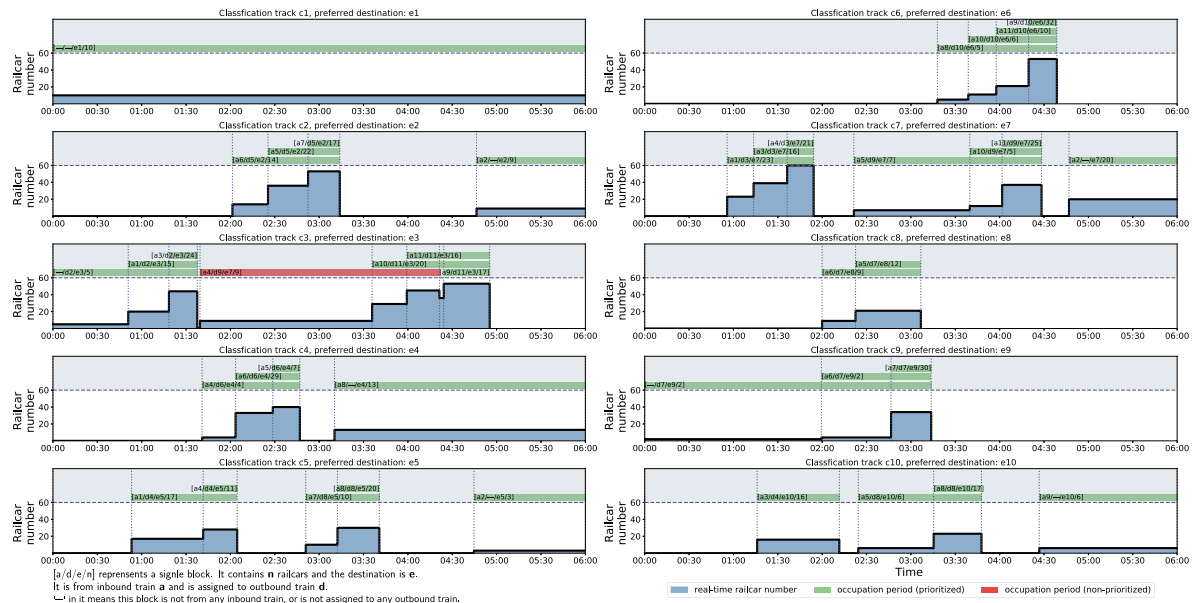


Fig. 11. The classification track utilization in the solution for the real-world instance of scenario M1-R1.

outbound train is determined and assembly starts. However, if the assembly starts later, more blocks could be assembled in this outbound train. An example is shown in Fig. 12, which is the block occupation records of classification track $c16$ in the solution to the real-world instance of scenario M2-L. After block $[a3/d4/e12/23]$ rolls into classification track $c16$ at 01:40, the current three blocks on it containing in total 52 railcars with destination $e12$ are determined to be the formation of outbound train $d4$. Then the assembly of $d4$ starts. If the assembly of $d4$ starts after the next block $[a5/-/e12/8]$ stops on the track at 02:11, the block $[a5/-/e12/8]$ can also be assembled to form outbound train $d4$, instead of failing to depart in this planning period. Then the average railcar dwell time can be further reduced.

However, due to the sequential decision-making nature of the DRL approach, i.e., the decisions of hump sequencing and classification track assignment are computed sequentially in the MDP process, the proposed DRL method is unable to predict the later transitions only based on the current state and action. Therefore, to guarantee an outbound train can depart on time, once the minimum railcar number limit is reached by the currently available blocks, the heuristic rule must determine the specific formation and assembly operations for the outbound train. In contrast, the MINLP model is modelled based on decision variables for assembly sequence and assembly start time simultaneously, which provides the possibility to wait for more railcars to assemble an outbound train with more railcars and thus further reduce the average railcar dwell time. Hence, the solutions obtained from the DRL method

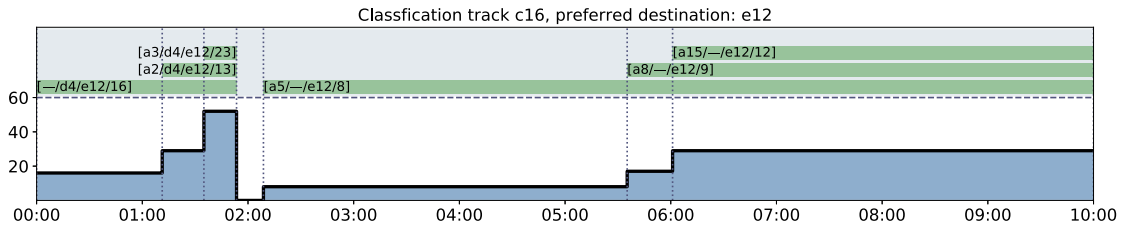


Fig. 12. The block occupation records on classification track c16 in the solution for the basis instance of scenario M2-L.

may assemble fewer railcars than the solutions from the decom-MINLP model. This is the major reason for the gap between the results of the DRL method and the MINLP model. Such a gap increases under the large-size scenario M2-L, compared to scenario M1-R1, as the railcars that are not assigned to any outbound trains might have longer dwell time due to the longer planning period. Whereas, it is caused by the inability of the heuristic rule of assembly to consider the succeeding blocks under a state, and does not indicate the degrading solving ability of the DRL method on larger instances.

Lastly, we need to point out that, although solving the decom-MINLP model by CPLEX can deliver better-quality solutions, our DRL method may be more applicable for many practical application requirements. Because, the computational efficiency of the RIO-MY problem is also a significant issue, while our DRL method can reduce about 75% solving time compared to the decom-MINLP mode (see Table 8). Specifically, the requirement of solving the RIO-MY problem multiple times in practical scenarios induces the concern for computational efficiency. For example, if an inbound train is delayed to arrive at the marshalling yard, the previously solved solution could be infeasible. The delay time could change multiple times until the arrival time is determined, leading to repeatedly solving the RIO-MY problem. Furthermore, when a solution is being executed, if any outbound trains fail to be assembled due to disassembly disruptions, the railcars in the classification yard input for the next solving stage will be changed, so a new solution to the next stage is required correspondingly. Meanwhile, disassembly disruptions may also result in the infeasibility of the solution being executed, requiring solving the RIO-MY problem again with real-time train operation situations and railcar distributions. The high-quality solutions and high computational efficiency of the DRL method enable us to satisfy such real-time re-solving requirements, which are detailed in Section 5.5. In summary, given the above facts that the RIO-MY problem is required to be solved repeatedly or in real-time, our DRL method could be a more practical option for decision-making in real-world operations.

5.4. Generalization on M1-R2 and M2-H

We now investigate the ability of the proposed DRL method in solving unseen scenarios. To this end, the trained DRL agent using the instances of scenario M1-R1 is applied directly to solve the instances of scenario M1-R2 without retraining. Similarly, the DRL agent trained with M2-L is used to directly solve the instances of scenario M2-H. Recall that in comparison to M1-R1, M1-R2 is also a regular-size scenario of marshalling yard M1 but its inbound and outbound trains have totally different characteristics. Compared to M2-L, M2-H is an even larger scenario of marshalling yard M2, consisting of 27 inbound trains and 26 outbound trains in 12 h.

The results of this set of generalization experiments are shown in Table 12. We observe that in general, the trained agents have a good generalization capability. For the synthetic testing instances of scenario M1-R2, the solutions returned by the DRL method outperform the compared heuristics, with an average performance gap of 5.00% to the decom-MINLP model. For the real-world instance, the gap is even smaller, i.e., 2.50%. These results demonstrate that the agent trained on M1-R1 can be transferred to M1-R2 and obtain high-quality solutions, without any retraining. Furthermore, the solving time of the DRL method on M1-R2 is also significantly shorter than the heuristic algorithms and the decom-MINLP model.

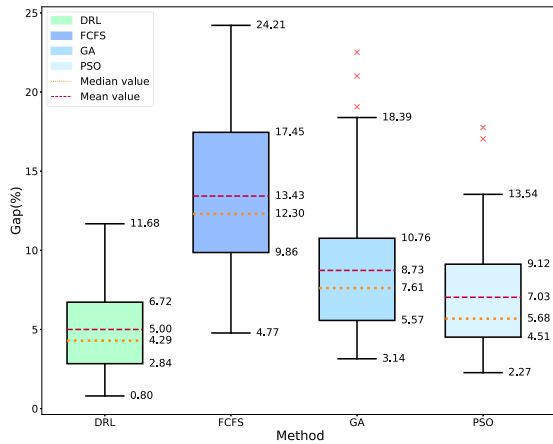
For the synthetic testing instances of the huge-size scenario M2-H, the average gap between the solutions returned from the DRL agent trained with M2-L and the decom-MINLP model is 11.64%. For the real-world instance, the gap between the DRL method and the decom-MINLP model is 12.95%. Such two gaps are enlarged compared to the results on scenario M2-L due to the same reason stated in the previous Section 5.3. That is, the dwell time of railcars not assigned to any outbound trains is increased. Nevertheless, these two gaps are still significantly smaller than the FCFS rule and the heuristic algorithms. Especially for the synthetic testing instances, the DRL method can decrease the average gap by about 9% compared to the FCFS rule (i.e., 11.64% vs. 20.39%), while the heuristic algorithms can only obtain about 2%–3% decrement of the average gap, which is similar to the results of M2-L.

In addition, the DRL agents consistently produce good solutions. As shown in Fig. 13, among the solutions for the 50 testing instances of M1-R2, 75% of them have a gap of less than 6.72%. In contrast, the solutions obtained by GA and PSO show large fluctuations with 3 and 2 outliers, respectively. Moreover, for the huge-size instances of M2-H, 75% of solutions from the DRL method have a gap of less than 13.92%. The solutions from the heuristic algorithms not only have limited improvement compared to the FCFS rule but also have performance outliers that are more than 30%. In conclusion, both agents trained on scenario M1-R1 and M2-L are able to produce stable and good solutions when generalizing them to other unseen scenarios of the same marshalling yards.

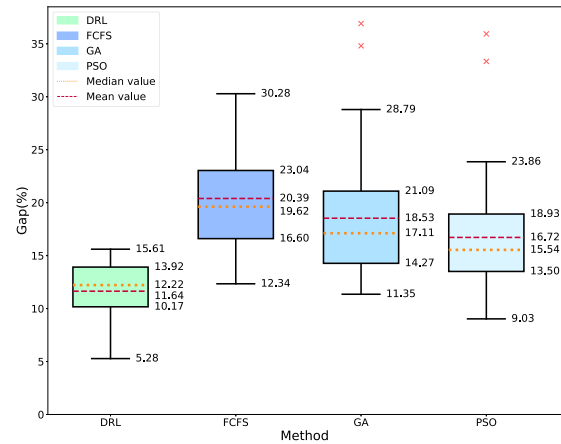
Another interesting outcome is, although the heuristic rule of assembling outbound trains is the major reason for the gap between the DRL method and the decom-MINLP model, it provides a possibility to assemble more outbound trains, especially when the

Table 12
Generalization performance on the scenario M1-R2 and M2-H.

Scenario	Indicator	DRL	FCFS	GA	PSO	decom-MINLP
M1-R2	F_{test}	12 484 s	13 482 s	12 925 s	12 724 s	11 890 s
	Gap_{test}	5.00%	13.43%	8.73%	7.03%	–
	Q_{ave}	8.25 s	0.03 s	104.15 s	106.85 s	40.98 s
	F_{real}	12 243 s	13 224 s	12 852 s	12 696 s	11 944 s
	Gap_{real}	2.50%	10.72%	7.60%	6.30%	–
M2-H	F_{test}	22 836 s	24 630 s	24 249 s	23 883 s	20 462 s
	Gap_{test}	11.64%	20.39%	18.53%	16.72%	–
	Q_{ave}	46.30 s	0.11 s	497.69 s	535.88 s	162.33 s
	F_{real}	23 281 s	25 553 s	24 522 s	24 187 s	20 612 s
	Gap_{real}	12.95%	23.97%	18.97%	17.35%	–



(a) Testing instances of scenario M1-R2



(b) Testing instances of scenario M2-H

Fig. 13. The gap distributions compared to the MINLP model solutions.

planning period is longer. For the real-world instance of the huge-size scenario M2-H, the average railcar dwell time in the solution from the decom-MINLP model is 20 462 s. Correspondingly, it cancels 5 outbound trains, and 21 outbound trains are assembled and depart on time. On the other hand, the solution from the DRL method has a larger railcar dwell time (24 187 s) but only cancels 4 outbound trains. One more outbound train is assembled and departs. Because, in such a long planning period of 12 h in M2-H, the decom-MINLP model assigns railcars to earlier outbound trains as much as possible according to the purpose of minimizing railcar dwell time. It probably incurs that some later outbound trains have no sufficient available railcars to form them, and they are therefore cancelled. In contrast, in the DRL method, an outbound train will not wait for more railcars to assemble due to the heuristic rule of assembling outbound trains. Hence, although some railcars are not assembled to form earlier outbound trains, they could still be resources for assembling later outbound trains. Therefore, more outbound trains could be assembled in the solutions from the DRL method. With more outbound trains, the transport capacity of the railway mainlines can be utilized more efficiently.

5.5. Extension: Handling disassembly disturbances

The metaheuristics, i.e., GA and PSO, and the decom-MINLP model require complete information to solve the RIO-MY problem. In contrast, the DRL method models and solves the problem as a sequential decision problem, which makes it easier to adapt to uncertainties and randomnesses during solving. We demonstrate this via a case where disturbances occur during disassembly.

In practice, during the process of disassembling an inbound train, a disturbance that a block rolls into an inconsistent track with the assigned classification track could occur due to human error. The disturbed disassembly process must be recovered by sending the hump shunting locomotive to pull back the block in the wrong classification track, then the block can roll into the assigned track correctly again. It is a very time-consuming process, which would postpone the operations of disassembling the rest blocks of this inbound train and humping all succeeding inbound trains. Therefore, after the disturbance, the original classification track assignment and consequent train makeup plan might be no longer feasible. Thanks to the sequential decision process of the DRL method, our method can continue to solve the instance by inputting the completed part of the episode after an unexpected disassembly disturbance occurs, without modification and retraining of the model. The completed actions before the disturbance do

Table 13

The new solution for affected inbound and outbound trains after the disassembly disturbance.

	Inbound train	Humping sequence index	Humping start time	Disassembly end time	Outbound train	Actual formation	Assembly sequence index	Assembly start time	Assembly end time
Original solution	<i>a2</i>	11	04:31	04:46	<i>d9</i>	[<i>a4/e7/9</i>], [<i>a5/e7/7</i>], [<i>a10/e7/5</i>], [<i>a11/e7/25</i>]	8	04:06	04:42
	<i>a7</i>	6	02:33	02:52					
	<i>a8</i>	7	02:57	03:17	<i>d10</i>	[<i>a8/e6/5</i>], [<i>a10/e6/6</i>], [<i>a11/e6/10</i>], [<i>a9/e6/32</i>]	9	04:23	04:50
	<i>a9</i>	10	04:06	04:26					
	<i>a10</i>	8	03:22	03:39	<i>d11</i>	[<i>a10/e3/20</i>], [<i>a11/e3/16</i>], [<i>a9/e3/17</i>]	10	04:42	05:08
	<i>a11</i>	9	03:44	04:01					
New solution	<i>a2</i>	11	04:54	05:09	<i>d9</i>	[<i>a4/e7/9</i>], [<i>a5/e7/7</i>], [<i>a11/e7/25</i>]	7	04:06	04:42
	<i>a7</i>	6	02:33	03:15					
	<i>a8</i>	7	03:20	03:40	<i>d10</i>	[<i>a8/e6/5</i>], [<i>a11/e6/10</i>], [<i>a9/e6/32</i>]	8	04:23	04:50
	<i>a9</i>	9	04:07	04:27					
	<i>a10</i>	10	04:32	04:49	<i>d11</i>	[<i>a11/e3/16</i>], [<i>a9/e3/17</i>], [<i>a10/e3/20</i>]	9	04:47	05:13
	<i>a11</i>	8	03:45	04:02					

not need decisions again from the DRL agent, so the solving process will be accelerated again, which enables the DRL method to provide new solutions for the disturbed disassembly process in seconds.

We use the real-world instance of the regular-size scenario M1-R1, i.e., the instance in Section 5.3.2, in the experiment. The inbound trains are identical to those in Table 10. A disassembly disturbance occurs to the block [*e5/10*] of inbound train *a7*. After recovering, the end time of disassembling *a7* is postponed from 02:52 to 03:15. The inbound trains after *a7* in the humping sequence are also postponed to be disassembled. If we continue to implement the original solution, such delays lead to infeasibility: when block [*e3/17*] containing 17 railcars from inbound train *a9* is disassembled, the classification track *c3* assigned to it only has 15 capacity. Hence, the original solution is infeasible and the train operations have to be terminated. Outbound trains *d5*, *d9*, *d10*, and *d11* have to be cancelled due to the terminated assembly operations.

We illustrate how the same DRL agent trained on scenario M1-R1 would resolve this disturbed instance. Based on completed actions before the disassembly disturbance, the agent continues to solve this instance to provide a new solution for trains affected by the disassembly disturbance. In Table 13, the upper half is the original solution for the affected inbound and outbound trains (collected from Tables 10 and 11); the new solution for affected trains are shown in the lower half, with highlighted changes in bold texts. Compared to the original solution, the new solution adjusts the humping sequence for inbound train *a9*, *a10*, and *a11*. Inbound train *a11* is humped first, which enables outbound train *d9* to reach the minimum railcar number limit (i.e., 40 railcars) by absorbing block [*a11/e7/25*] earlier and abandoning block [*a10/e7/5*]. Then outbound train *d9* is assembled, so that block [*a4/e7/9*] parking on the non-prioritized classification track *c3* (see Fig. 11) is pulled out, which releases capacity on classification track *c3* for railcars with destination *e3* from inbound train *a9* and *a10*. Such reasonable sequencing and assignment eliminate the infeasible classification track assignment after the disassembly disturbance in the original solution. Hence, in this new solution, the outbound train *d9*, *d10*, and *d11* can be assembled again. The average railcar dwell time is 14 048 s. Therefore, the proposed DRL method can still obtain a new high-quality solution after a disassembly disturbance occurs without extra modelling, training, or modification.

Table 14 shows the experiment results of 10 instances, including the above real-world instance (No. 1 in the table) and 9 synthetic testing instances (No. 2–10 in the table) of scenario M1-R1. For each instance, a stochastic disassembly disturbance is simulated. After the disturbance, the original solution is kept to be implemented if it is still feasible. Otherwise, starting from the infeasible action, we apply the FCFS rule to obtain a new feasible solution. For the DRL method, it provides a new solution by using the trained DRL agent to make new decisions from the disturbed disassembly step to replace the original solution. The experiment results show that the disassembly disturbance usually renders the original solutions infeasible, as the postponed blocks fail to be assembled and the classification tracks therefore have no sufficient capacity for succeeding blocks. For these infeasible instances, the new solutions provided by the DRL method outperform the FCFS rule. For example, for the real-world instance (No. 1 in Table 14), the results of average railcar dwell time in new solutions are 14 048 s and 14 465 s using DRL and FCFS, respectively. Hence, after the disassembly disturbance, the DRL method can eliminate infeasibility for postponed blocks and trains and reduce the railcar dwell time simultaneously. Furthermore, even if the original solution is still feasible after the disassembly disturbance (i.e., instances of No. 3, 7, and 8), applying the DRL method to obtain a new solution is still possible to reduce the railcar dwell time. Note we did not compare with the GA and PSO algorithms and the decom-MINLP model, since they would have to solve the instances with new situations completely, in case that the original solutions become infeasible due to disturbances.

6. Conclusion

This paper proposes a graph neural network (GNN) based deep reinforcement learning (DRL) framework to address the railcar itinerary optimization in the railway marshalling yard (RIO-MY). We formulate train operation processes in the marshalling yard as a Markov decision process (MDP). The state is encoded as a tripartite graph consisting of inbound train nodes, outbound train nodes, classification track nodes, and edges among nodes to reflect traffic dynamics in the marshalling yard. A joint action combines

Table 14
The results for handling disassembly disturbance in multiple instances.

No.	Average railcar dwell time (seconds)		
	Original solution	New solution - FCFS (after infeasible action)	New solution - DRL (after disassembly disturbance)
1	Infeasible	14 465	14 043
2	Infeasible	14 737	14 315
3	14 618	–	14 618
4	Infeasible	15 617	14 757
5	Infeasible	14 727	14 511
6	Infeasible	15 818	15 678
7	14 749	–	14 599
8	14 902	–	14 857
9	Infeasible	14 107	13 837
10	Infeasible	15 214	14 706

the decisions on humping sequencing and classification track assignment into one action. A waiting action is added to assist the humping sequencing. To complete state transitions, a discrete-event simulation model of train operations in the marshalling yard is developed to serve as the environment of DRL, in which the heuristic rules of outbound train assembly and shunting locomotive scheduling are integrated. The reward function aiming to minimize the average railcar dwell time estimates the railcar dwell time under a state to specify the reward in each step. To reason out action based on the graph encoded state, we parameterize the policy by a tripartite graph neural network (TGIN) and a decision-making network (DMN). The TGIN is designed to learn informative node embeddings on the tripartite graph by messages passing and aggregation. The DMN leverages node embeddings to sample actions according to action probabilities under each state. Finally, we use the Proximal Policy Optimization (PPO) algorithm to train and optimize the policy network. Through a comprehensive set of numerical experiments, this study extends our knowledge of the learning based railcar itinerary optimization in the marshalling yard in the following aspects.

- The proposed DRL method can produce high-quality solutions for instances of the RIO-MY problem. Under the two scenarios covering 4 h and 8 hours (M1-R1 and M2-L), the solutions for testing instances have average gaps of only 3.93% and 7.52% compared to the baseline solutions provided by solving the MINLP model, respectively, and outperform the FCFS rule and heuristic algorithms.
- A trained DRL agent can be generalized to unlearned scenarios without retraining while maintaining good performance. The solutions obtained by trained agents for the testing instance of the two new scenarios (M1-R2 covering 4 h and M2-H covering 12 h) only have average gaps of 5.00% and 11.64% compared to the baseline solutions.
- The DRL method has high solving efficiency. For the same testing instances, the average solving time using the DRL method is significantly less than the heuristic algorithms. Even if solving the decomposed MINLP model using a commercial solver, the average solving time is still about 4 times of the DRL method.
- A disturbed disassembly process can be recovered using a trained DRL agent promptly without extra development. A new high-quality solution can be provided to re-optimize the affected railcar itineraries.
- The proposed tripartite graph and the graph neural network (i.e., TGIN) are the key components of the DRL method. Using TGIN to generate node embeddings on the tripartite graph extracts informative and effective features to enable more accessible learning for the DRL algorithm. It also relieves the heavy work of state feature engineering and reward shaping.

We believe this is the first study that applies the emerging deep reinforcement learning techniques to train marshalling related problems. Although the proposed method shows promising capability for the RIO-MY problem, it still has some limitations. In particular, the discrete-event simulation model, which is used to be the environment in the DRL framework, simplifies the train movement processes to running time parameters. So, the specific route occupations are not considered in detail. Even though the running time parameters refer to real-world data and some measures are added to deal with potential route conflicts (see [Appendix C](#)), such a simplification may still result in infeasibility to the original solutions. In addition, some special requirements are not considered, such as the re-humping operations and the designated railcar order when assembling an outbound train. Moreover, the generalization of the proposed method should be limited in the same marshalling yard if retraining is not conducted. Due to the differences in yard layouts, locomotive specifications, operational protocols, and railcar characteristics across various marshalling yards, it is required to adjust the simulation model parameters and retrain the agent to ensure the solution quality of the proposed method's in new marshalling yards.

We close by pointing to several possible directions for further research. The most obvious next step is that a multi-agent DRL framework, replacing the heuristic rules in this study with agents for assembly determination and shunting locomotive scheduling, could improve solutions closer to the MINLP model. However, it also means the MDP should be modelled by time step, leading to more difficulties. Furthermore, if the outbound train needs to assemble railcars according to a specific order of their destinations, a more sophisticated DRL agent is required, which can be treated as an independent problem. Last but not least, the train routing and scheduling problem in railway marshalling yards are also expected to be included in the simulation model, instead of simplified by

running time parameters. This can provide more microscopic details in the planning stage, enabling a more robust operation plan for subsequent real-time operations.

CRedit authorship contribution statement

Hongxiang Zhang: Writing – original draft, Visualization, Validation, Methodology, Investigation, Conceptualization. **Gongyuan Lu:** Writing – review & editing, Supervision, Project administration, Investigation, Funding acquisition, Data curation. **Yingqian Zhang:** Writing – review & editing, Supervision, Software, Resources, Project administration, Conceptualization. **Andrea D’Ariano:** Writing – review & editing, Supervision, Resources. **Yaoxin Wu:** Writing – review & editing, Supervision, Methodology, Investigation, Conceptualization.

Acknowledgements

This work is supported by National Key Research and Development Program of China (No. 2022YFB4300504), National Natural Science Foundation of China (No. 52272324), and Natural Science Foundation of Sichuan Province (No. 2022NSFSC0397). The first author is deeply grateful for the support from the China Scholarship Council, China (No. 202207000095). The generous support and valuable discussion of our colleagues Bojian Zhang and Zili Shen from Southwest Jiaotong University, Peng Chen from Guangzhou Municipal Engineering Design & Research Institute Co., Ltd., and Shuang Zhang from Chang’an University are greatly appreciated.

Appendix A. The heuristic rule for assembling outbound trains

The heuristic rule of assembling outbound trains is applied to determine the specific formation and sequence for assembling outbound trains. The basic logic of it is to check which outbound trains can be assembled, and select available blocks according to the railcar dwell time, as shown in Algorithm A.1. The algorithm is called by the DRL agent during each transition, exactly when the execution of the selected action δ_i is completed.

Specifically, after the selected action δ_i is executed under the state S_i , outbound trains are checked one by one according to the order of their scheduled departure time (Line 1). If an outbound train has been already assembled or cancelled, it will be skipped (Line 2–4). Considering the required movement time for assembly (t_s^{lm} and t_s^{lr}) and the departure technical inspection time (t_d^{tech}), an outbound train is cancelled if it cannot depart on time (Line 5–8).

When an outbound train d is checked valid, the attempt to assemble this outbound train is started. For each classification track c , the blocks currently parking on it (Φ_c^i) are checked by traversing from the end near the departure yard for assembly, since a classification track is a FIFO queue (Line 10–11). If the destination of block β meets the formation requirements of outbound train d , then block β is available to assemble outbound train d and thus saved in the set X (Line 12–13). Otherwise, block β cannot be assembled for outbound train d and it obstructs the blocks parking behind it on the same classification track. Hence, the checking on classification track c can be terminated (Line 14–16).

After iterating all classification tracks, all available blocks are saved in set X . Then, the blocks are sorted according to the ascending order of the arrival time of the inbound trains they are from (Line 19). By traversing blocks in this sorted order (Line 21), the railcars with longer dwell time currently can be leveraged first to assemble outbound train d , so that the overall railcar dwell time is expected to be reduced. For each block β , if assemble it does not cause the total number of railcars to exceed the maximum number limit λ_d^{max} , it can be determined to form outbound train d (Line 22–24). Otherwise, the block β cannot be selected and obstructs other available blocks that park behind β on the same classification track. Therefore, block β and blocks obstructed by β are removed from set X (Line 25–27).

After all blocks in set X are checked, the blocks that are determined to form the outbound train d are saved in set Y . If they can reach the minimum number limit λ_d^{min} (Line 29), the number of classification tracks that all determined blocks park on are counted to calculate the pull-out time t_d^{po} (Line 30–31). Simultaneously, the heuristic rule for scheduling pull-out shunting locomotives is applied here to obtain the earliest time that a pull-out shunting locomotive is available to serve the assembly task (Line 32). Then, the end time of assembling outbound train d can be calculated. By adding the time of departure technical inspection, if the total time is not later than the scheduled departure time, we can now determine to assemble outbound train d using blocks in Y (Line 33–36).

Algorithm A.1 Heuristic rules for outbound train assembly

Require: τ^i , the current system time of state S_i ; $d \in D$, the outbound trains; $\Phi_c^i, \forall c \in C$, the blocks parking on the classification track c in current state S_i after the action δ_i is executed.

```

1: for all  $d \in D$  do
2:   if  $d$  has been assembled or cancelled then
3:     continue
4:   end if
5:   if  $\tau^i + t_s^{lm} + t_s^{lr} + t_d^{tech} \geq t_d^{out}$  then
6:     cancel outbound train  $d$ 
7:     continue
8:   end if
9:    $X = \emptyset$ 
10:  for all  $c \in C$  do

```

▷ Traverse in order of scheduled departure time

```

11:   for all  $\beta \in \Phi_c^i$  do
12:     if  $\beta$  meets the formation requirements of  $d$  then
13:       Add  $\beta$  to  $X$ 
14:     else
15:       break
16:     end if
17:   end for
18: end for
19: Sort all blocks in  $X$  in ascending order of arrival time.
20:  $Y = \emptyset, count = 0$ 
21: for all  $\beta \in X$  do
22:   if  $count + \text{the number of railcars in } \beta \leq \lambda_d^{max}$  then
23:     Add  $\beta$  to  $Y$ 
24:      $count += \text{the number of railcars in } \beta$ 
25:   else
26:     Remove all blocks parking behind block  $\beta$  on the same classification track from the set  $X$ 
27:   end if
28: end for
29: if  $count \geq \lambda_d^{min}$  then
30:   Count the number of involved classification tracks of blocks in  $Y$ .
31:   Calculate the required time of assembly ( $t_d^{po}$ )
32:   Let  $\tau^{zz}$  be the earliest time that any pull-out shunting locomotive becomes available, i.e., the heuristic rule of scheduling pull-out shunting locomotives.
33:   if  $\max\{\tau^i, \tau_s^{zz}\} + t_s^m + t_d^{po} + t_s^{rr} + t_d^{tech} \leq t_d^{out}$  then
34:     Determine  $d$  to be assembled
35:     break
36:   end if
37: end if
38: end for

```

Ensure: The outbound train d to be assembled and blocks assigned to it, if applicable.

Appendix B. Raw features

The raw features of nodes and edges in the tripartite graph G_i representing state S_i are defined as follows (see Tables B.1–B.4).

Appendix C. The simulation model of the railway marshalling yard

A discrete-event simulation model serves as the environment of the proposed DRL method. It is a widely used method to depict various train operations (Zhang et al., 2023, 2024; Li et al., 2023). In this paper, the simulation model is developed by the SimPy package (<https://simpy.readthedocs.io/en/latest/>) in Python. It accomplishes simulation by controlling multiple processes in parallel to trigger the state change events of systems and entities. Coordinating with the DRL policy network, the simulation model of the railway marshalling yard is shown as Fig. C.1.

A main process is set to simulate the system clock, which is advanced in the mode of next state time. Interacting with the DRL policy network and the MDP model of our RIO-MY problem, the state in the discrete-event simulation model is at the same moments as the states of the MDP model. A state will be encoded as the tripartite graph and input to the DRL policy, and an action selected by the DRL policy will be input for the simulation model to advance the system to the next state.

At the beginning of the simulation, all events of inbound train arrival are scheduled according to the train timetable. The processes of arrival technical inspection are triggered by the arrival events. The start state S_0 is at the moment that the first inbound train finishes the arrival technical inspection.

Table B.1

Raw features of inbound train nodes, 66 dimensions in total.

Feature	Dimension	Content and value
Status	1	Not arrive (−1); has arrived, but humping for it has not started (0); humping has started or has ended (1).
Arrival index	1	The index of arrival specified by the train timetable.
Formation	60	The destinations of each railcar in the inbound train, destinations are indexed from 1. If the inbound train contains less than 60 railcars, fill the rest with 0.
Humping index	1	The index of humping this inbound train. The value is 0 if the humping has not started.
Humping start time	1	The time that the assigned hump shunting locomotive begins to push this inbound train.
Index of block being disassembled	1	The index of the block currently being disassembled in relation to all blocks of this inbound train; if disassembly has not started, the value is 0; if disassembly is in process, the value is the current index; if disassembly has been completed, the value is −1.
Real-time number of railcars	1	The number of railcars in the inbound train, which will change after the humping and disassembly start.

Table B.2

Raw features of outbound train nodes, 68 dimensions in total.

Feature	Dimension	Content and value
Status	1	Not assembled (0); assembly has been determined (1); cancelled (−1);
Formation requirement	5	The required destinations for railcars to form the outbound train; if the number of required destinations is less than 5, fill the rest with 0.
Assembly start time	1	The time that the assigned pull-out shunting locomotive for assembling the outbound train begins to move to the classification yard.
Assembly end time	1	The time that the outbound train has been transferred to and parked on the berthing track in the departure yard.
Determined formation	60	The destinations of each railcar assigned to form the outbound train; if the total number of railcars is less than 60, fill the rest with 0; if the outbound train has been cancelled or the assembly for it has not been determined, fill all elements with 0.

Table B.3

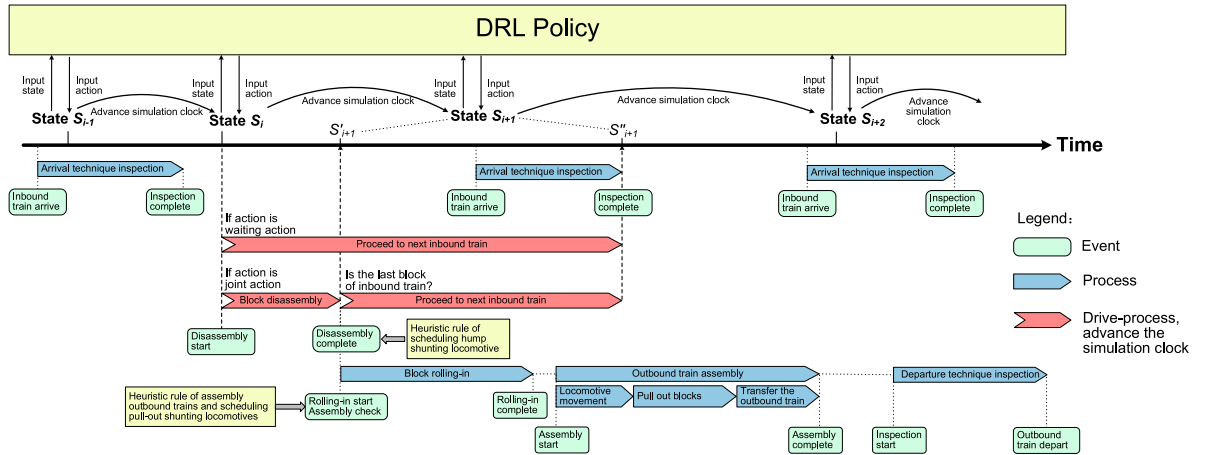
Raw features of classification track nodes, 121 dimensions in total.

Feature	Dimension	Content and value
Preferred destination	1	The preferred destination of the classification track. Destinations are indexed from 1, same as railcar formation in Tables B.1 and B.2.
Current railcars parking on	60	The destination of railcars currently parking on the classification track; if the number of current railcars is less than 60, fill the rest with 0.
Source inbound train of current railcars	60	The arrival index of the inbound train that contains the railcar currently parking on the classification track; if the number of current railcars is less than 60, fill the rest with 0.

Table B.4

Raw features of edges, 2 dimensions in total.

Feature	Dimension	Content and value
Number of passed railcars	1	The number of railcars that are assigned from the inbound train to the classification track, or from the classification track to the outbound train.
Is added or not	1	if the edge is built at the start state, = 1; if the edge is added during transitions, = 0.

**Fig. C.1.** The discrete-event simulation model of the railway marshalling yard.

Under a state S_i , the sub-processes to be the basis of advancing the simulation clock to the next state, named drive-process, are created according to the selected action. Specifically, if the selected action is the *waiting action*, then a drive-process is created to advance the simulation model to the moment that the next inbound train finishes its arrival technical inspection, and the simulation model transits to the next state $S_{i+1} = S''_{i+1}$. The reward of this step is computed at the end of the drive-process. In contrast, if the selected action is an *joint action*, then a drive-process of disassembling a block from the specific inbound train is created. If the block is not the last block of the inbound train currently being disassembled, the drive-process will advance the simulation clock to the next state $S_{i+1} = S'_{i+1}$, which is at the moment that disassembling the current block is completed. Otherwise, if the block is the last block and no inbound train in the receiving yard is ready to be humped, one more drive-process is added, which advances the simulation model to the moment that the next inbound train finishes its arrival technical inspection. The state transits to the next

state $S_{i+1} = S''_{i+1}$ correspondingly. Please note that the heuristic rule of scheduling hump shunting locomotives is applied here to consider locomotive availability in determining the moment of the next state.

Other sub-processes are created and executed in parallel to complete the train operation simulation. The sub-process of rolling the block into the assigned classification track is created once the drive-process of disassembling the block is completed. At the same time, the heuristic rule of assembling the outbound trains and scheduling the pull-out shunting locomotives is applied here to determine whether any process of assembling outbound trains needs to be created after the rolling-in process. Based on the results of assembly determination, the reward of the drive-process step is also computed here. If necessary, the sub-processes of assembling an outbound train, including the assigned pull-out shunting locomotive moving to the classification yard, pulling out blocks from classification tracks, and transferring the assembled outbound train to the departure yard, are created after the rolling-in is completed. The sub-process of departure technique inspection is then scheduled to trigger the event that the outbound train departs at the scheduled departure time. While advancing the simulation clock by drive-processes, the above sub-processes are executed synchronously, and the related events are triggered automatically.

Here, we provide more details of the simulation model for potential concerns. These details are some settings and constraints in the simulation model to enhance the robustness and applicability of the proposed method, especially regarding route conflicts and classification yard safety.

- (1) The train (railcar) movements in the marshalling yard are simplified to running time parameters in the simulation model. That is, the duration of a movement is determined by the given running time parameter, instead of modelling track details and microscopic train kinematics. As we stated before, the state transitions are determined according to the time required for a drive-process. Other sub-processes are also related to train movements. Hence, based on this simplification, the MDP model is lightweight without train running details and focuses on RIO-MY problem related decisions. On the other hand, we collect real-world data of train running time and set them as parameters for train movements to make the simulation model closer to reality. Meanwhile, to better describe the real-world scenario, the data is detailed down to the microscopic level. For a train movement, the origination track, the destination track, the number of railcars in the train, and the operation type, together determine the running time parameter value.
- (2) To allocate more time for potential route conflicts, the running parameters of train movements are scaled up. Since the train movements are simplified, route conflicts are not simulated in the model. When applying the solutions of the RIO-MY problem in real-world operations, routes scheduled for train movements could lead to conflicts. Hence, to reserve more time in advance for trains to wait due to route conflicts, we scale up the corresponding running time parameters. For example, in the experiment scenarios, two shunting locomotives can be used to pull out railcars and assemble outbound trains. We double the value of running time parameters of assembly-related movements in the case that two assembly operations are executed simultaneously. In doing so, even if a pulling-out operation has to wait for the other one due to route conflicts, it can be guaranteed to be completed in the allocated time. For other running time parameters, including humping movements, train arrival and departure movements, etc., we enlarge the values by 1.2–1.4 times. The above scale-ups for running time parameters are expected to provide more backup time for train movements in real-time operations.
- (3) A five-minute time margin is set between two consecutive humping operations, aiming to avoid potential route conflicts in the hump area. After an inbound train has been disassembled on the hump completely, we add a time interval (5 min) before the next inbound train begins to be humped. In this period, the shunting locomotive of the previous inbound train can leave the hump area without a potential route conflict with the movements of humping the next inbound train.
- (4) Rolling-in and pulling-out operations cannot be executed at the same time on both ends of a classification track. To ensure safety, in real-world operations, if a block is rolling into a classification track, pulling out railcars from the other end of the classification track simultaneously is forbidden, and vice versa. To this end, in the simulation model, we give a higher priority to assembly operations than the disassembly operations. This means if a pulling-out operation on a classification track is in process and the disassembled block is assigned to the same classification track, the disassembly operation will be held temporarily and continue after the pull-out operation completes.
- (5) The capacity limits of the receiving yard and departure yard are set according to the number of berthing tracks. To avoid that there are no available berthing tracks for inbound trains and outbound trains, we set the capacity limits of the receiving yard and departure yard in the simulation model. In the training process of the DRL agent, if the number of trains exceeds the capacity limits, the episode will be terminated immediately, as we stated in Section 4.2.6.

Appendix D. Benchmark: the simulation–optimization based heuristic algorithms

For benchmarking with metaheuristics, we develop a simulation–optimization framework to solve the RIO-MY problem, which is also a widely used method (Hassannayebi et al., 2014; Wang et al., 2023a; Högdahl and Bohlin, 2023; Liu et al., 2022). The discrete-event simulation model developed in Appendix C and heuristic algorithms are integrated to develop the framework, as shown in Fig. D.2. The heuristic algorithms include the genetic algorithm(GA) and the particle swarm optimization (PSO), which are programmed using the *scikit-opt* package (<https://scikit-opt.github.io/>) in Python. The simulation model is used to be the fitness function for heuristic algorithms, enabling easy implementation of heuristic algorithms without programming the entire RIO-MY problem in the algorithms.

In this framework, the heuristic algorithms only focus on the two crucial decisions, i.e., the humping sequence and the classification track assignment. That is, the heuristic algorithms play the same role as the DRL agent in this study, so that a fair

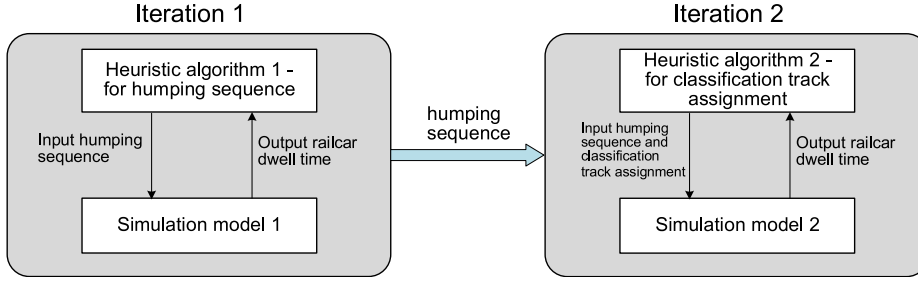


Fig. D.2. The simulation-optimization framework based on heuristic algorithms.

benchmarking can be implemented. Considering that integrating the two crucial decisions in a single heuristic algorithm will lead to a vast search space and difficulties in encoding decisions, a two-stage workflow is adopted to implement the heuristic algorithms for the two crucial decisions. Specifically, the humping sequence is solved first by the heuristic algorithm. The simulation model developed in Appendix C is applied here as the fitness function of the heuristic algorithm. A minor modification for the simulation model is removing the capacity limits of classification tracks. It means all blocks can always be assigned to their corresponding prioritized classification tracks, no classification track assignment is required. Therefore, the heuristic algorithm provides the humping sequences as the input of the simulation model, the simulation model outputs the average railcar dwell time as the fitness value for the heuristic algorithm to guide its iterative search process. After a given number of iterations, the ideal humping sequence with the minimum railcar dwell time is obtained and input for the next phase of solving, i.e., the classification track assignment. In this second phase circulation, the simulation model includes the capacity limit of classification tracks again, identical to the one in Appendix C. Based on the given humping sequence and the capacity limit of classification tracks, the number and content of the blocks that require non-prioritized classification tracks are known. Therefore, the heuristic algorithm provides the classification track assignments, which are input together with the given humping sequences to the simulation model. The simulation model outputs the average railcar dwell time as the fitness value for the heuristic algorithm. The final results of railcar dwell times are obtained after a given number of iterations. Notably, the two heuristic algorithms in this two-stage framework will adopt the same one in benchmarking, i.e., both use GA or both use PSO. The population size is set to 30, and the iteration limit is set to 150.

Appendix E. Benchmark: the decomposition method to solve the MINLP model

To provide baseline solutions for benchmarking, we solve the MINLP model stated in Section 3.2 using the commercial solver Gurobi. However, solving the original model of a real-world instance remains a considerable computational challenge due to the non-linear characteristics leading to millions of variables. Therefore, we apply a decomposition method to solve the original MINLP model. Notably, the decomposed models involving train makeup plan are still developed using railcar as the decision unit, so that railcars can be assigned flexibly to minimize railcar dwell time. The train operation parameters in the MINLP model are set as: train humping — 20 min; train disassembly — 22 min; train assembly — 30 min.

Specifically, the first sub-model is to assign railcars to outbound trains without the constraints of classification track assignments, therefore, an ideal train makeup plan with minimum railcar dwell time can be obtained. However, in a solution from this sub-model, railcars may not comply with the first-in-first-out principle, leading to many obstructions among railcars when assembling them on the classification tracks. Therefore, based on the humping sequence and assembly sequence output by the first sub-model, the second sub-model aims to adjust the train makeup plan to guarantee that, for railcars with the same destination, the ones entering the classification yard first will also be assembled first. The result of the second sub-model is the adjusted train makeup plan, based on which railcars can be grouped into blocks. Hence, together with the results of the first sub-model, the enter and exit time of each block at the classification yard are obtained to be the input of the third sub-model, i.e., the classification track assignment sub-model. This sub-model assigns classification tracks to blocks according to the capacity limits and the principle regarding the prioritized classification track. Solving these three sub-models sequentially, the finalized railcar itineraries with the minimized railcar dwell time can be obtained. We formulate these three sub-models as follows.

(1) **Sub-model1** is formulated as:

$$\min \sum_{b \in B} \sum_{d \in D} (u_d^{out} - u_{ab}^{in}) y_b^d \quad (E.1)$$

Constraints of inbound train humping: Eqs. (1) to (5);

Constraints of train makeup plan: Eqs. (6) to (13);

Constraints of outbound train assembly: Eqs. (20) to (25).

In this sub-model, the train makeup plan, the humping sequence, the assembly sequence, and the shunting locomotive schedules are determined. The classification track assignment is ignored. Therefore, the objective function Eq. (E.1) only computes the railcar dwell time. The constraints are inherited from the original MINLP model. The results of this sub-model contain an ideal train makeup

plan with the minimum railcar dwell time. The corresponding humping sequence, assembly sequence, and shunting locomotive activities are also obtained.

(2) **Sub-model2** adjusts the train makeup plan based on the results of sub-model1. The primary purpose of the adjustment is to guarantee railcars with the same destination will be assembled in the same order they enter the classification yard, which guarantees blocks are units for disassembly and assembly and facilitates the classification track assignment for blocks. The results of sub-model1 have already specified the humping sequence, the assembled outbound trains, and the assembly sequence. Hence, the rest work in this sub-model only needs to assign railcars according to the given start and end time of humping and assembly, the limits of railcar numbers, and the formation requirements. Sub-model2 is formulated as:

$$\min \sum_{b \in B} \sum_{d \in D} (u_d^{out} - u_{ab}^{in}) y_b^d \quad (E.2)$$

$$\sum_{d \in D} y_b^d \leq 1, \quad \forall b \in B \quad (E.3)$$

$$y_b^d \leq z_b^d, \quad \forall d \in D, b \in B \quad (E.4)$$

$$\lambda_d^{min} \theta_d \leq \sum_{b \in B} y_b^d \leq \lambda_d^{max}, \quad \forall d \in D \quad (E.5)$$

$$y_b^d \leq \theta_d, \quad \forall d \in D, b \in B \quad (E.6)$$

$$\sum_{d \in D} \tau_d^{sn} y_b^d \leq \sum_{d' \in D} \tau_{d'}^{sn} y_{b'}^{d'} \quad b, b' \in B \text{ and } p_{a_b, a_{b'}} = 1 \text{ and } e_b = e_{b'} \quad (E.7)$$

$$y_b^d = 0 \quad d \in D, b \in B_a, a \in A \text{ and } \tau_d^{sn} < \tau_a^{he} \quad (E.8)$$

In this sub-model, the objective function is the same as Eq. (E.1), aiming to minimize railcar dwell time. Only y_b^d is the decision variable. The constraints Eqs. (E.3) to (E.6) are identical to Eqs. (6) to (9) in the original MINLP model, by which the number and destination of railcars assigned to an outbound train are guaranteed to satisfy the formation requirements. Here θ_d is the known value specified by the results of sub-model1. Besides, two new constraints Eqs. (E.7) and (E.8) are added. In Eq. (E.7), for any two railcars b and b' , if their destinations are the same (i.e., $e_b = e_{b'}$) and railcar b rolls into the classification yard before railcar b' (i.e., $p_{a_b, a_{b'}} = 1$), then railcar b must not be assembled later than railcar b' , as the inequality in Eq. (E.7) shows. Hence, railcars with the same destination will be assembled in the order that they enter the classification yard, so that the FIFO principle can be guaranteed. Eq. (E.8) is the time constraint of assigning railcars to outbound trains based on the humping and assembly sequence results of sub-model1. If the disassembly end time of inbound train a is later than the assembly start time of outbound train d , i.e., $\tau_d^{sn} \leq \tau_a^{he}$, then all railcars from inbound train a cannot be assigned to outbound train d .

By solving sub-model2, the train makeup plan is adjusted to guarantee reasonable orders for railcars to be assembled. Such adjustments only switch railcar assignments among outbound trains based on the results of the assembly sequence and formation of outbound trains inherited from sub-model1. Therefore, the railcar dwell time obtained by sub-model2 is close to the ideal results of sub-model1. Based on the results of sub-model2, the railcars can be grouped into blocks. The train makeup plan and the time of a block entering and exiting the classification yard are also obtained, which are input to the next sub-model.

(3) **Sub-model3** is to assign classification tracks to blocks based on the capacity limits and preferred destinations of classification tracks. The blocks are specified by the results of sub-model2. We define the set of blocks as B indexed by β , i.e., $\beta \in B$. It can replace b and B in classification track assignment constraints. The sub-model3 is formulated as:

$$\min \sum_{\beta \in B} \sum_{c \in C} g_{\beta}^c y_{\beta}^d k_{\beta} \quad (E.9)$$

$$\sum_{c \in C} x_{\beta}^c = 1, \quad \forall \beta \in B \quad (E.10)$$

$$\sum_{a \in A} \sum_{\beta \in B_a} x_{\beta}^c p_{a', a} k_{\beta} + \sum_{\beta \in B_a} x_{\beta}^c k_{\beta} - \sum_{d \in D} \sum_{\beta \in B} x_{\beta}^c y_{\beta}^d e_a^d k_{\beta} \leq \kappa_c, \quad \forall a \in A, c \in C \quad (E.11)$$

$$\sum_{d \in D} \tau_d^{sn} y_{\beta}^d - \sum_{d' \in D} \tau_{d'}^{sn} y_{\beta'}^{d'} \leq M(2 - x_{\beta}^c - x_{\beta'}^c), \quad \forall c \in C, \beta, \beta' \in B \text{ and } p_{a_{\beta}, a_{\beta'}} = 1 \quad (E.12)$$

In this sub-model, the objective function Eq. (E.9) is to minimize the number of railcars assigned to non-prioritized classification tracks. Only x_{β}^c is the decision variable. Similar to Eq. (17), constraint Eq. (E.11) guarantees the number of railcars in blocks assigned to a classification track does not exceed the capacity limit. k_{β} is the number of railcars in block β . With the known humping sequence and assembly sequence, constraint Eq. (E.12) guarantees the first-in-first-out principle of blocks on a classification track, which is similar to Eqs. (15) and (16) in the original MINLP model.

Integrating the results of the above three sub-models, railcar itineraries and the integrated train operation plan with minimized railcar dwell time can be obtained. Based on this decomposition method, the solving time of the MINLP model using Gurobi is reduced significantly to a few minutes.

References

- Agasucci, V., Grani, G., Lamorgese, L., 2023. Solving the train dispatching problem via deep reinforcement learning. *J. Rail Transp. Plan. Manage.* 26, 100394.
- Bektaş, T., Crainic, T.G., Morency, V., 2009. Improving the performance of rail yards through dynamic reassignments of empty cars. *Transp. Res. C (ISSN: 0968-090X)* 17 (3), 259–273.
- Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S., 2016. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*.
- Bohlin, M., Dahms, F., Flier, H., Gestrelus, S., 2012. Optimal freight train classification using column generation. In: 12th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2012). Vol. 25, Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, pp. 10–22.
- Bohlin, M., Gestrelus, S., Dahms, F., Mihalák, M., Flier, H., 2016. Optimization methods for multistage freight train formation. *Transp. Sci.* 50 (3), 823–840.
- Bohlin, M., Hansmann, R., Zimmermann, U.T., 2018. Optimization of railway freight shunting. *Handb. Optim. Railw. Ind.* 181–212.
- Boysen, N., Emde, S., Flidner, M., 2016. The basic train makeup problem in shunting yards. *OR Spectr.* 38, 207–233.
- Boysen, N., Flidner, M., Jaehn, F., Pesch, E., 2012. Shunting yard operations: Theoretical aspects and applications. *European J. Oper. Res.* 220 (1), 1–14.
- Boysen, N., Flidner, M., Jaehn, F., Pesch, E., 2013. A survey on container processing in railway yards. *Transp. Sci.* 47 (3), 312–329.
- Cappart, Q., Chételat, D., Khalil, E.B., Lodi, A., Morris, C., Velickovic, P., 2023. Combinatorial optimization and reasoning with graph neural networks. *J. Mach. Learn. Res.* 24, 130–131.
- Cappart, Q., Goutier, E., Bergman, D., Rousseau, L.-M., 2019. Improving optimization bounds using machine learning: Decision diagrams meet deep reinforcement learning. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33, pp. 1443–1451.
- Chen, X., Guo, X., Meng, J., Xu, R., Li, S., Li, D., 2023. Research on ATO control method for urban rail based on deep reinforcement learning. *IEEE Access* 11, 5919–5928.
- de Oliveira da Costa, P.R., Rhuggenaath, J., Zhang, Y., Akcay, A., Lee, W.-J., Kaymak, U., 2020. Data-driven policy on feasibility determination for the train shunting problem. In: *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2019*. Springer, pp. 719–734.
- Dahlhaus, E., Horak, P., Miller, M., Ryan, J.F., 2000. The train marshalling problem. *Discrete Appl. Math.* 103 (1–3), 41–54.
- Di Stefano, G., Koči, M.L., 2004. A graph theoretical approach to the shunting problem. *Electron. Notes Theor. Comput. Sci.* 92, 16–33.
- Drori, I., Kharkar, A., Sickinger, W.R., Kates, B., Ma, Q., Ge, S., Dolev, E., Dietrich, B., Williamson, D.P., Udell, M., 2020. Learning to solve combinatorial optimization problems on real-world graphs in linear time. In: 2020 19th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 19–24.
- Duan, L., Hu, H., Qian, Y., Gong, Y., Zhang, X., Xu, Y., Wei, J., 2018. A multi-task selected learning approach for solving 3D flexible bin packing problem. *arXiv preprint arXiv:1804.06896*.
- Haahr, J.T., Lusby, R.M., 2016. A matheuristic approach to integrate humping and pullout sequencing operations at railroad hump yards. *Networks* 67 (2), 126–138.
- Hassannayebi, E., Sajedinejad, A., Mardani, S., 2014. Urban rail transit planning using a two-stage simulation-based optimization approach. *Simul. Model. Pract. Theory* 49, 151–166.
- He, S., Song, R., Chaudhry, S.S., 2000. Fuzzy dispatching model and genetic algorithms for railyards operations. *European J. Oper. Res.* 124 (2), 307–331.
- He, S., Song, R., Chaudhry, S.S., 2003. An integrated dispatching model for rail yards operations. *Comput. Oper. Res.* 30 (7), 939–966.
- Högdahl, J., Bohlin, M., 2023. A combined simulation-optimization approach for robust timetabling on main railway lines. *Transp. Sci.* 57 (1), 52–81.
- Hu, H., Zhang, X., Yan, X., Wang, L., Xu, Y., 2017. Solving a new 3d bin packing problem with deep reinforcement learning method. *arXiv preprint arXiv:1708.05930*.
- Jacob, R., Márton, P., Maue, J., Nunkesser, M., 2011. Multistage methods for freight train classification. *Networks* 57 (1), 87–105.
- Jaehn, F., Rieder, J., Wiehl, A., 2015a. Minimizing delays in a shunting yard. *OR Spectrum* 37, 407–429.
- Jaehn, F., Rieder, J., Wiehl, A., 2015b. Single-stage shunting minimizing weighted departure times. *Omega* 52, 133–141.
- Khadilkar, H., 2018. A scalable reinforcement learning algorithm for scheduling railway lines. *IEEE Trans. Intell. Transp. Syst.* 20 (2), 727–736.
- Khalil, E., Dai, H., Zhang, Y., Dilkina, B., Song, L., 2017. Learning combinatorial optimization algorithms over graphs. *Adv. Neural Inf. Process. Syst.* 30.
- Kool, W., van Hoof, H., Welling, M., 2018. Attention, learn to solve routing problems!. In: *International Conference on Learning Representations*.
- Kraft, E.R., 2000. A hump sequencing algorithm for real time management of train connection reliability. In: *Journal of the Transportation Research Forum*. Vol. 39, pp. 95–115.
- Kraft, E.R., 2002a. Priority-based classification for improving connection reliability in railroad yards-part I: Integration with car scheduling. In: *Journal of the Transportation Research Forum*. Vol. 41, pp. 93–105.
- Kraft, E.R., 2002b. Priority-based classification for improving connection reliability in railroad yards-part II: Dynamic block to track assignment. In: *Journal of the Transportation Research Forum*. Vol. 41, pp. 107–119.
- Kwon, Y.-D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S., 2020. Pomo: Policy optimization with multiple optima for reinforcement learning. *Adv. Neural Inf. Process. Syst.* 33, 21188–21198.
- Lee, W.-J., Jamshidi, H., Roijers, D.M., 2020. Deep reinforcement learning for solving train unit shunting problem with interval timing. In: *Dependable Computing-EDCC 2020 Workshops: A14RAILS, DREAMS, DSOGRI, SERENE 2020*, Munich, Germany, September 7, 2020, Proceedings 16. Springer International Publishing, pp. 99–110.
- Lei, K., Guo, P., Zhao, W., Wang, Y., Qian, L., Meng, X., Tang, L., 2022. A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem. *Expert Syst. Appl.* 205, 117796.
- Li, H., Jin, M., Song, R., He, S., Song, J., 2016. Dynamic railcar connection planning in classification yards. *Transp. Lett.* 8 (2), 77–89.
- Li, W., Ni, S., 2022. Train timetabling with the general learning environment and multi-agent deep reinforcement learning. *Transp. Res. B* 157, 230–251.
- Li, H., Song, R., Jin, M., He, S., 2014. Optimization of railcar connection plan in a classification yard. In: *Transportation Research Board 93rd Annual Meeting*. Transportation Research Board, pp. 14–3091.
- Li, S., Yan, Z., Wu, C., 2021. Learning to delegate for large-scale vehicle routing. *Adv. Neural Inf. Process. Syst.* 34, 26198–26211.
- Li, Z., Zhang, H., Wen, W., He, B., Wang, Y., Lu, G., 2023. Evaluating car-to-train assignment strategies for the railway marshalling yard using a multi-agent simulation approach. In: 10th International Conference on Railway Operations Modelling and Analysis-RailBelgrade 2023. p. 125.
- Lin, Z., Wu, Y., Zhou, B., Cao, Z., Song, W., Zhang, Y., Jayavelu, S., 2024. Cross-problem learning for solving vehicle routing problems. *arXiv preprint arXiv:2404.11677*.
- Liu, R., Dai, X., Cui, D., Yue, P., 2022. Real-time rescheduling for multi-section high-speed railway trains based on simulation optimization. In: 2022 IEEE 25th International Conference on Intelligent Transportation Systems. ITSC, IEEE, pp. 2967–2973.
- Ma, Y., Li, J., Cao, Z., Song, W., Guo, H., Gong, Y., Chee, Y.M., 2022. Efficient neural neighborhood search for pickup and delivery problems. In: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*. pp. 4776–4784.
- Maue, J., Nunkesser, M., 2009. Evaluation of Computational Methods for Freight Train Classification Schedules. ARRIVAL Project.
- Mazyavkina, N., Sviridov, S., Ivanov, S., Burnaev, E., 2021. Reinforcement learning for combinatorial optimization: A survey. *Comput. Oper. Res.* 134, 105400.
- Park, J., Chun, J., Kim, S.H., Kim, Y., Park, J., 2021. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *Int. J. Prod. Res.* 59 (11), 3360–3377.

- Peer, E., Menkovski, V., Zhang, Y., Lee, W.-J., 2018. Shunting trains with deep reinforcement learning. In: 2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC). IEEE, pp. 3063–3068.
- Raut, S., Sinha, S.K., Khadilkar, H., Salsingkar, S., 2019. A rolling horizon optimisation model for consolidated hump yard operational planning. *J. Rail Transp. Plan. Manage.* 9, 3–21.
- Ruf, M., Cordeau, J.-F., 2021. Adaptive large neighborhood search for integrated planning in railroad classification yards. *Transp. Res. B* 150, 26–51.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Šemrov, D., Marsetič, R., Žura, M., Todorovski, L., Srdic, A., 2016. Reinforcement learning approach for train rescheduling on a single-track railway. *Transp. Res. B* 86, 250–267.
- Shi, T., Zhou, X., 2015. A mixed integer programming model for optimizing multi-level operations process in railroad yards. *Transp. Res. B* 80, 19–39.
- Smit, I.G., Zhou, J., Reijnen, R., Wu, Y., Chen, J., Zhang, C., Bukhsh, Z., Nuijten, W., Zhang, Y., 2024. Graph neural networks for job shop scheduling problems: A survey. *arXiv preprint arXiv:2406.14096*.
- Song, W., Chen, X., Li, Q., Cao, Z., 2023. Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Trans. Ind. Inform.* 19 (2), 1600–1610.
- Su, S., Wang, X., Tang, T., Wang, G., Cao, Y., 2021. Energy-efficient operation by cooperative control among trains: A multi-agent reinforcement learning approach. *Control Eng. Pract.* 116, 104901.
- Tang, Y., Agrawal, S., Faenza, Y., 2020. Reinforcement learning for integer programming: Learning to cut. In: *International Conference on Machine Learning*. PMLR, pp. 9367–9376.
- Trepat Borecka, J., Bešinović, N., Maknoon, Y.M., Goverde, R.M., Lee, W.-J., 2021. Solving the train unit shunting problem using multi-agent deep reinforcement learning with routing optimization. In: 2021 INFORMS Annual Meeting.
- van de Ven, A., Zhang, Y., Lee, W.-J., 2019a. Boosting local search using machine learning: a study on improving local search by graph classification in determining capacity of shunting yards. In: *Agents and Artificial Intelligence: 11th International Conference, ICAART 2019, Prague, Czech Republic*. Springer, pp. 183–203.
- van de Ven, A., Zhang, Y., Lee, W.-J., Eshuis, R., Wilbik, A., 2019b. Determining capacity of shunting yards by combining graph classification with local search. In: *ICAART (2)*. pp. 285–293.
- Wang, X., D'Ariano, A., Su, S., Tang, T., 2023b. Cooperative train control during the power supply shortage in metro system: A multi-agent reinforcement learning approach. *Transp. Res. B* 170, 244–278.
- Wang, H., Peng, T., Nassehi, A., Tang, R., 2023a. A data-driven simulation-optimization framework for generating priority dispatching rules in dynamic job shop scheduling with uncertainties. *J. Manuf. Syst.* 70, 288–308.
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Philip, S.Y., 2020. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* 32 (1), 4–24.
- Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A., 2021. Learning improvement heuristics for solving routing problems. *IEEE Trans. Neural Netw. Learn. Syst.* 33 (9), 5057–5069.
- Wu, Y., Zhou, J., Xia, Y., Zhang, X., Cao, Z., Zhang, J., 2023. Neural airport ground handling. *IEEE Trans. Intell. Transp. Syst.* 1–15.
- Xu, Y., Fang, M., Chen, L., Xu, G., Du, Y., Zhang, C., 2021. Reinforcement learning with multiple relational attention for solving vehicle routing problems. *IEEE Trans. Cybern.* 52 (10), 11107–11120.
- Xu, K., Hu, W., Leskovec, J., Jegelka, S., 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826*.
- Yagar, S., Saccomanno, F., Shi, Q., 1983. An efficient sequencing model for humping in a rail yard. *Transp. Res. A* 17 (4), 251–262.
- Yin, J., Chen, D., Li, L., 2014. Intelligent train operation algorithms for subway by expert system and reinforcement learning. *IEEE Trans. Intell. Transp. Syst.* 15 (6), 2561–2571.
- Ying, C., Chow, A.H., Chin, K.-S., 2020. An actor-critic deep reinforcement learning approach for metro train scheduling with rolling stock circulation under stochastic demand. *Transp. Res. B (ISSN: 0191-2615)* 140, 210–235.
- Ying, C.-S., Chow, A.H.F., Wang, Y.-H., Chin, K.-S., 2022. Adaptive metro service schedule and train composition with a proximal policy optimization approach based on deep reinforcement learning. *IEEE Trans. Intell. Transp. Syst.* 23 (7), 6895–6906.
- Zhang, H., Guo, M., Hu, L., Lu, G., 2024. A multi-agent simulation based train platforming research for facilitating passenger transfer in a high-speed railway station. *Simul. Model. Pract. Theory (ISSN: 1569-190X)* 130, 102856.
- Zhang, H., Lu, G., Lei, Y., Zhang, G., Niyitanga, I., 2023. A hybrid framework for synchronized passenger and train traffic simulation in an urban rail transit network. *Int. J. Rail Transp.* 11 (6), 912–941.
- Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Chi, X., 2020a. Learning to dispatch for job shop scheduling via deep reinforcement learning. In: *Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (Eds.), Advances in Neural Information Processing Systems. Vol. 33, Curran Associates, Inc.*, pp. 1621–1632.
- Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Chi, X., 2020b. Learning to dispatch for job shop scheduling via deep reinforcement learning. *Adv. Neural Inf. Process. Syst.* 33, 1621–1632.
- Zhang, Y., Song, R., He, S., Li, H., Guo, X., et al., 2018. Optimization of classification track assignment considering block sequence at train marshaling yard. *J. Adv. Transp.* 2018.
- Zhang, L., Zhou, M., Li, Z., et al., 2021. An intelligent train operation method based on event-driven deep reinforcement learning. *IEEE Trans. Ind. Inform.* 18 (10), 6973–6980.
- Zhao, J., Dick, C.T., 2023. Quantifying the influence of volume variability on railway hump classification yard performance with AnyLogic simulation. *Transp. Res. Rec.* 03611981231160178.
- Zhou, J., Wu, Y., Cao, Z., Song, W., Zhang, J., Chen, Z., 2023. Learning large neighborhood search for vehicle routing in airport ground handling. *IEEE Trans. Knowl. Data Eng.* 35 (9), 9769–9782.
- Zhu, Q., Su, S., Tang, T., Liu, W., Zhang, Z., Tian, Q., 2022. An eco-driving algorithm for trains through distributing energy: A Q-learning approach. *ISA Trans.* 122, 24–37.
- Zhu, Y., Wang, H., Goverde, R.M., 2020. Reinforcement learning in railway timetable rescheduling. In: 2020 IEEE 23rd International Conference on Intelligent Transportation Systems. ITSC, IEEE, pp. 1–6.